



Birzeit University

Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ENCS5300 – Graduation Project

Lumen: A Task-Oriented Assistive System for Blind People

Prepared by:

Diaa Badaha – 1210478

Ahmad Hamdan – 1210241

Omar Husein – 1212738

Supervised by: Dr. Mohammad Husein

Section: 9

A Graduation Project submitted to the Department of Electrical and
Computer Engineering in partial fulfillment of the requirements for the
degree of B.Sc. in Computer Engineering

Birzeit

July, 2026

Abstract

Visual impairment affects hundreds of millions of individuals worldwide, and indoor environments remain among the most difficult settings for independent mobility: satellite positioning is unavailable, layouts are unfamiliar, and traditional aids such as the white cane provide no information beyond arm’s reach. Existing assistive vision systems either narrate the scene continuously, overwhelming the user, or answer isolated queries, forcing the user to know what to ask for in a space they cannot see. This work presents **Lumen**, a task-oriented assistive system that occupies the middle ground: perception activates only in response to an explicit spoken task, provides structured guidance until the task completes, and then returns to silence.

Lumen requires no dedicated hardware and no application install. The user opens a web page on an ordinary smartphone; the browser streams camera frames, push-to-talk voice commands, and compass headings over a single WebSocket to a Python backend that runs the entire perception and speech stack: faster-whisper for speech recognition, a fuzzy command parser tolerant of transcription errors, YOLOv8 object detection, MediaPipe hand tracking, and server-side speech synthesis. A finite state machine governs every session, making task activation, completion, cancellation, and error recovery explicit and testable.

The system supports two task families. *Object Allocation* locates a requested object and guides the user toward it with direction and per-class distance cues; at close range it enters a reach-guidance phase that steers the user’s hand toward the target and completes the task automatically when the fingertip reaches it. *Navigation* implements goal-directed exploration: the user names only a destination room, and Lumen guides them door-by-door using a compass-anchored 360° room scan, a three-layer door-perception funnel built around a custom-trained door detector, a pinhole distance model that speaks in steps, and an obstacle watchdog that fuses named-object detection with class-agnostic floor segmentation while the user walks.

The decision logic of both task families is isolated from models and I/O and is verified by an automated suite of 281 tests, including scripted multi-room navigation journeys, while live trials in real apartments drove the perception design failure-by-failure. Lumen demonstrates that structured, task-scoped assistance can be delivered on commodity hardware while respecting the cognitive load, autonomy, and safety of blind users.

المستخلص

يؤثر ضعف البصر على مئات الملايين من الأشخاص حول العالم، وتبقى البيئات الداخلية من أصعب الأماكن للتنقل المستقل: فأنظمة تحديد المواقع غير متاحة داخل المباني، والمخططات غير مألوفة، والوسائل التقليدية مثل العصا البيضاء لا تقدم معلومات تتجاوز متناول الذراع. تميل أنظمة الرؤية المساعدة الحالية إما إلى وصف المشهد وصفاً متواصلاً يُغرق المستخدم بسيل سمعي دائم، أو إلى الإجابة عن استفسارات منفردة تُجبر المستخدم على معرفة ما يسأل عنه في مكان لا يستطيع رؤيته. يقدم هذا العمل نظام «لومن»، وهو نظام مساعد موجه بالمهام يحتل المنطقة الوسطى: إذ يُفَعِّل الإدراك البصري فقط استجابةً لمهمة منطوقة صريحة، ويقدم إرشاداً منظماً حتى اكتمال المهمة، ثم يعود إلى الصمت.

لا يتطلب «لومن» أي عتاد مخصص ولا تثبيت أي تطبيق؛ إذ يفتح المستخدم صفحة ويب على هاتف ذكي عادي، فيبث المتصفح إشارات الكاميرا والأوامر الصوتية واتجاه البوصلة عبر اتصال وب سوكِت واحد إلى خادم بايثون يشغل كامل منظومة الإدراك والكلام: التعرف على الكلام، ومحلل أوامر متسامح مع أخطاء النسخ، وكشف الأجسام بنموذج يولو، وتتبع اليد، وتوليد الكلام من جهة الخادم. وتُحكم آلة حالات منتهية كل جلسة، مما يجعل تفعيل المهام وإكمالها وإلغاءها والتعافي من الأخطاء أموراً صريحة وقابلة للاختبار.

يدعم النظام عائلتين من المهام: «تحديد موقع الأجسام» التي تجد الجسم المطلوب وترشد المستخدم نحوه بإشارات الاتجاه والمسافة، وتدخل عن قرب في طور «إرشاد الوصول» الذي يوجه يد المستخدم نحو الهدف ويُكمل المهمة تلقائياً عند ملامسة أطراف الأصابع له، و«الملاحة» التي تطبق الاستكشاف الموجه بالهدف: يذكر المستخدم اسم الغرفة المقصودة فقط، فيرشده «لومن» من باب إلى باب عبر مسح دائري للغرفة مثبت بالبوصلة، وكشف الوصول الدلالي من الأجسام الدالة على الغرفة، ومنظومة إدراك أبواب ثلاثية الطبقات مبنية حول كاشف أبواب مدرب خصيصاً، ونموذج مسافة يتحدث بالخطوات، وكشف عبور المدخل، وحارس عوائق يدمج كشف الأجسام المسماة مع تقسيم أرضية المشهد أثناء سير المستخدم.

جرى عزل منطق القرار في عائلتي المهام عن النماذج والمدخلات والمخرجات، ويجري التحقق منه عبر مجموعة آلية من 281 اختباراً تشمل رحلات ملاحة متعددة الغرف. وقد وُجِّهت التجارب الحية في شقق حقيقية تصميم الإدراك خطوةً بخطوة. يبرهن «لومن» على أن المساعدة المنظمة المحددة بالمهام يمكن تقديمها على عتاد شائع مع احترام الحمل المعرفي واستقلالية وسلامة المستخدمين المكفوفين.

Contents

English Abstract	i
Arabic Abstract	ii
Table of Contents	iii
List of Tables	vii
List of Figures	viii
List of Abbreviations	ix
1 Introduction and Motivation	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 High-Level Methodology	3
1.4 Contributions	4
1.5 Scope	4
1.6 Report Outline	5
2 Background	6
2.1 Vision-Based Perception	6
2.1.1 Object Detection	6
2.1.2 Relative Spatial Reasoning	7
2.1.3 Semantic Segmentation and Monocular Depth	7
2.1.4 Hand Pose Estimation	7
2.2 Speech-to-Text (STT)	8
2.3 Language Understanding	8
2.4 Text-to-Speech (TTS)	8
2.5 Browser Device Sensing	9
2.6 State-Based System Control	9

3	Related Work	10
3.1	Assistive Vision Systems for Blind Users	10
3.2	Interaction Paradigms: Continuous Narration vs. On-Demand Assistance	11
3.3	Indoor Navigation and Object-Finding Systems	12
3.4	Cognitive Load and Information Management	12
3.5	Wearable Platforms and Voice-Based Interaction	13
3.6	System Architectures, Control Models, and Safety	13
3.7	Summary and Research Gap	14
4	System Design	15
4.1	From Proposal to Final Design	15
4.2	Requirements	16
4.2.1	Functional Requirements	16
4.2.2	Non-Functional Requirements	16
4.3	Architecture Overview	16
4.3.1	Wire Protocol	17
4.3.2	Session Model	18
4.4	The Task Finite State Machine	18
4.5	Voice Interaction Design	18
4.6	Safety Design	19
5	Task Families and Algorithms	20
5.1	Object Allocation	20
5.1.1	Detection and Temporal Filtering	20
5.1.2	Spatial Reasoning with Per-Class Size Priors	21
5.1.3	Guidance Policy	21
5.1.4	The Reach Guidance Phase	22
5.1.5	Information Queries	22
5.2	Navigation: Goal-Directed Exploration	23
5.2.1	Destination Model and Semantic Arrival	23
5.2.2	The Door Detector	24
5.2.3	The Three-Layer Door Perception Funnel	24
5.2.4	The Compass-Anchored Room Scan	25
5.2.5	The Exploration Controller	26
5.2.6	Door Approach: Distance in Steps	28
5.2.7	The Obstacle Watchdog	28
5.2.8	Doorway Transit Detection	29
5.2.9	Multi-Door Rooms	30
5.2.10	Speech Discipline	30

6	Implementation	32
6.1	Technology Stack	32
6.2	Code Organization at a Glance	32
6.3	The Wire Protocol	34
6.4	Backend Organization	34
6.5	The Voice Pipeline	36
6.5.1	The Command Parser	37
6.6	The Object Allocation Engine	39
6.7	The Navigation Engine: From Prototype to Integration	39
6.7.1	The Standalone Prototype	39
6.7.2	Integration	40
6.7.3	Package Layout and the Per-Frame Pipeline	40
6.8	Session Continuity and Haptics	41
6.9	Frontend	43
6.10	Deployment	44
6.11	Test Infrastructure	45
7	Evaluation	47
7.1	Door Detector Results	47
7.2	Offline Evaluation of the Command Parser	48
7.3	Automated Behavioral Verification	49
7.4	Live Trials	50
7.4.1	Methodology	50
7.4.2	Principal Findings and the Mechanisms They Produced	51
7.4.3	Qualitative Outcomes	51
7.5	Evaluation Against Requirements	53
7.6	System Performance	53
8	Limitations	56
8.1	Perception and Navigation	56
8.2	System and Deployment	57
8.3	Evaluation Scope	58
9	Future Work	59
9.1	Deepening the Human-in-the-Loop Navigation Design	59
9.2	Exploration Memory and Bounded Autonomy	60
9.3	Perception Robustness	60
9.4	User Studies	60
9.5	Platform Evolution	61

10 Conclusion	62
References	64

List of Tables

5.1	Destination rooms and indicator objects	23
6.1	Technology choices and their rationale.	33
6.2	Implementation scale	33
6.3	Binary frame tags	34
6.4	JSON control messages	34
6.5	FSM transition table	35
6.6	Per-class distance priors	36
6.7	Parser vocabulary and thresholds	38
6.8	Object Allocation tuning constants	39
6.9	Navigation package layout	41
6.10	Key navigation constants	42
6.11	Haptic vocabulary	43
6.12	Frontend modules	43
6.13	Model inventory	45
6.14	Test suite composition	46
7.1	Door detector validation results	47
7.2	Command parser offline evaluation	49
7.3	Live perception failures	51
7.4	Live guidance and speech failures	52
7.5	Requirements verification	54

List of Figures

4.1	System architecture	17
4.2	Session finite state machine	18
5.1	Door perception funnel	24
5.2	Navigation controller state machine	26

List of Abbreviations

ADE20K	A scene-parsing dataset used to train semantic segmentation models
AI	Artificial Intelligence
API	Application Programming Interface
COCO	Common Objects in Context (dataset)
CPU	Central Processing Unit
FOV	Field of View
FPS	Frames Per Second
FSM	Finite State Machine
GPS	Global Positioning System
GPU	Graphics Processing Unit
gTTS	Google Text-to-Speech
HTTPS	Hypertext Transfer Protocol Secure
IMU	Inertial Measurement Unit
IoU	Intersection over Union
JPEG	Joint Photographic Experts Group (image format)
JSON	JavaScript Object Notation
LRU	Least Recently Used (cache eviction policy)
mAP	mean Average Precision
MP3	MPEG-1 Audio Layer III (audio format)
PTT	Push-to-Talk
SLAM	Simultaneous Localization and Mapping
STT	Speech-to-Text
TTS	Text-to-Speech
WS	WebSocket
YOLO	You Only Look Once (object detection family)
YOLOv8n/s/m	YOLOv8 Nano / Small / Medium variants

Chapter 1

Introduction and Motivation

1.1 Motivation

Visual impairment affects approximately 2.2 billion people worldwide, including 39 million who are completely blind [1]. These numbers continue rising due to population growth and aging, making independent navigation a critical challenge for millions of individuals.

Among the most pressing difficulties is indoor navigation, which remains largely unsolved despite significant advances in outdoor GPS-based systems [2]. The inability to navigate independently in unfamiliar indoor spaces — offices, hospitals, shopping centers — severely limits employment opportunities, social participation, and overall quality of life for blind individuals.

Traditional mobility aids have inherent limitations. The white cane detects obstacles only within physical reach, missing overhead hazards and providing no advance warning [3]. Guide dogs, while effective, require extensive training, cost between \$15,000 and \$50,000, and are not suitable for all users due to various constraints [4]. Indoor environments further complicate navigation as building structures attenuate GPS signals, rendering them unreliable [5].

Recent electronic travel aids have attempted to address these challenges through continuous scene narration — constantly verbalizing environmental information. However, this approach introduces a critical problem: cognitive overload. Users must process continuous auditory streams while simultaneously attending to ambient sounds, tactile feedback, and spatial reasoning [6]. Research shows this can be overwhelming and counterproductive, causing users to miss critical details.

This tension between providing sufficient navigation information and avoiding cognitive overload motivates a different approach. Rather than continuous environmental description, a system can activate perception only when the user explicitly requests assistance for a specific task, guide that task to completion, and then fall silent. This task-oriented paradigm reduces cognitive load, preserves user control, and matches how

blind individuals actually operate indoors: they do not want a running commentary — they want to *find the cup, reach the kitchen, or grasp the object in front of them*.

1.2 Problem Statement

Current assistive navigation systems for blind users typically fall into two categories, each with fundamental limitations. The first category comprises continuous narration systems that constantly describe the surrounding environment through audio feedback. These systems attempt to provide comprehensive scene understanding by continuously verbalizing detected objects, spatial relationships, and environmental changes. While this approach appears thorough, it creates severe cognitive load problems: users must simultaneously process continuous auditory information while attending to ambient sounds, tactile feedback from their mobility aids, and spatial reasoning [7]. Research on cognitive load in assistive technologies has demonstrated that this constant information flow can overwhelm users, leading to reduced task performance and increased mental fatigue.

The second category consists of query-based systems where users must explicitly request information about specific aspects of their environment [5], [8]. While these systems reduce cognitive load by providing information only on demand, they introduce a different problem: users must know precisely what to ask for and when. In unfamiliar environments, blind users often cannot anticipate what information they need until it becomes critically necessary. This creates a reactive rather than proactive assistance paradigm, where users may miss important environmental cues simply because they did not know to query for them.

Both approaches share a common limitation: they treat perception as either continuously active or completely passive. Continuous systems waste computational resources and overwhelm users with unnecessary information; query-based systems place the burden of managing perception entirely on the user. Neither aligns with how blind individuals naturally navigate — performing specific tasks such as locating objects, reaching destinations, or avoiding obstacles, each with distinct information requirements and a clear beginning and end.

What remains missing is a middle ground: a system that activates perception and provides guidance in response to explicit user-initiated tasks, but does so intelligently without requiring users to specify every detail. The core research challenge is designing an interaction paradigm and system architecture that enables *task-scoped perception* — where visual processing and guidance activate only during explicitly requested tasks, provide structured assistance toward task completion, and return to an idle state once the task concludes. A second challenge follows directly from it: for the hardest task family, indoor navigation, the system must guide a user to a destination room *without maps, localization infrastructure, or prior knowledge of the building* — using only what a

phone camera sees and what the user can naturally contribute. This work addresses the research question: *How can assistive navigation systems provide reliable, context-aware guidance while minimizing cognitive load through task-oriented perception activation?*

1.3 High-Level Methodology

To address the gap identified above, Lumen introduces a task-oriented assistive vision system that activates perception only in response to explicit user commands. The user opens a web page on an ordinary smartphone; no dedicated hardware and no application install are required. The phone’s browser captures camera frames, push-to-talk voice commands, and compass headings, and streams them over a single WebSocket connection to a backend server that performs all computation: speech recognition, command parsing, object detection, hand tracking, spatial reasoning, and speech synthesis. Guidance returns to the user as synthesized voice, complemented by short haptic cues.

The initial project proposal envisioned a purpose-built wearable device. During development this was deliberately replaced by the smartphone-browser architecture: the phone already integrates the camera, microphone, speaker, compass, and wireless connectivity that the wearable would have had to replicate, and serving the client as a web page eliminates manufacturing, installation, and platform-approval barriers entirely (Section 4.1 details this evolution). The architectural principle is unchanged: a thin sensing client, with all intelligence in a server the client reaches over the network.

The system organizes assistance into two task families:

- **Object Allocation:** locates a requested item (“find my cup”) and guides the user toward it with direction and distance cues derived from per-class size priors. Once the object is within arm’s reach, the task enters its *reach guidance* phase: it tracks the user’s hand and steers it to the target (“move your hand to the right ... almost there”), completing the task automatically when the fingertip reaches the object.
- **Navigation:** guides the user to a destination room (“navigate to the kitchen”) through *goal-directed exploration*: the system scans each room with a compass-anchored 360° sweep, recognizes arrival from room-indicator objects (a refrigerator and an oven imply a kitchen), and otherwise walks the user to a detected door and through it, room by room, warning about obstacles along the walking path.

A finite state machine (FSM) manages the system’s operational states, ensuring perception activates only during explicit user tasks. Every transition is driven by an explicit event — a voice command, a recognized completion, a cancellation, or a detected success — making system behavior predictable, interpretable, and testable.

1.4 Contributions

The main contributions of this project are:

- **Task-Scoped Perception Paradigm:** a task-oriented perception paradigm that activates vision processing only during explicit user commands, eliminating continuous cognitive load while maintaining proactive assistance within each task.
- **Goal-Directed Exploration Navigation Without Localization:** an indoor navigation approach that requires no maps, markers, or SLAM. The user names only the destination; the system explores door-by-door, detects arrival semantically from room-indicator objects, and treats the user as a direction oracle only when perception is genuinely ambiguous.
- **A Safety-Hardened Door Perception Funnel:** a three-layer door detection pipeline — a custom-trained single-class door detector ($\text{mAP@50} = 0.946$), geometric gates against walls and whole-scene false positives, and a second 4-class model as a semantic verifier — designed failure-by-failure against real phone footage, because a false door points a blind user at a wall.
- **Zero-Install Commodity-Hardware Delivery:** the complete system runs from a phone browser against a single Python server over one WebSocket, with an iOS-compatible audio pipeline and tunnel-based HTTPS deployment in minutes.
- **FSM-Governed, Testable Assistive Architecture:** explicit state-machine control of every session, with the decision logic of both task families isolated from models and I/O and verified by 281 automated tests, including scripted multi-room navigation journeys.

1.5 Scope

Lumen targets indoor residential and office-like environments: rooms connected by doors, containing everyday objects from the COCO vocabulary. Object Allocation supports a curated set of common object classes; Navigation supports six destination room types recognized by their characteristic objects (kitchen, bathroom, bedroom, living room, dining room, office). The system complements — and explicitly does not replace — the white cane: its value is advance information in the 2–4 m range, not real-time collision avoidance. English voice interaction is supported. The demonstrated deployment serves one active user per task session, with the architecture designed for per-session isolation.

1.6 Report Outline

The remainder of this report is organized as follows. **Chapter 2** provides the foundational concepts behind the system: object detection, speech recognition and synthesis, language understanding, and state-based control. **Chapter 3** surveys assistive vision systems, interaction paradigms, indoor navigation approaches, and control architectures, positioning Lumen within the research landscape. **Chapter 4** presents the system architecture: the client–server design, the wire protocol, the session model, and the finite state machine, including the design’s evolution from the original wearable proposal. **Chapter 5** details the two task families and their algorithms — Object Allocation, Reach Guidance, and goal-directed exploration Navigation, including the door detector training and the obstacle watchdog. **Chapter 6** describes the implementation: the technology stack, the per-session task engines, the speech pipeline, deployment, and the automated test infrastructure. **Chapter 7** evaluates the system through model metrics, automated behavioral verification, and live trials, and discusses limitations. **Chapter 10** concludes and outlines future work.

Chapter 2

Background

This chapter introduces the technical building blocks Lumen is assembled from: visual perception (object detection, spatial reasoning, segmentation, and depth), the voice pipeline (speech recognition, command understanding, speech synthesis), device sensing available to a web browser, and state-based system control.

2.1 Vision-Based Perception

2.1.1 Object Detection

Vision-based object detection enables Lumen to perceive and interpret the surrounding environment by identifying relevant objects and estimating their spatial location from camera input. Modern object detectors process an input image to simultaneously predict object class labels and corresponding bounding boxes, allowing real-time localization and recognition within a single inference pipeline [9]. Among these approaches, one-stage detectors such as the You Only Look Once (YOLO) family are particularly suitable for assistive systems due to their low inference latency and unified detection formulation, which directly maps images to object predictions without intermediate proposal stages [10]. The YOLOv8 family [11] spans a range of model sizes trading accuracy for speed, all pretrained on the COCO dataset [12], whose 80 everyday object classes (cup, chair, refrigerator, bed, ...) align well with the vocabulary of indoor assistive tasks.

Lumen uses two YOLOv8 variants for two different jobs. **YOLOv8n** (nano, ~6 MB) serves Object Allocation, where the target classes are large in frame and the detection loop must run at ~5 FPS even on CPU. **YOLOv8m** (medium) serves Navigation, where the decisive factor is recall on room-indicator objects seen at an angle and across a room — detection accuracy, not inference time, is the bottleneck there. In addition, Navigation uses two custom-trained YOLOv8 door models (Section 5.2.2), since COCO contains no door class.

2.1.2 Relative Spatial Reasoning

Bounding boxes alone do not tell a blind user anything actionable; they must be translated into egocentric guidance. Lumen infers the target’s horizontal *region* (left / center / right) from the box centre, and a coarse *distance* bucket (near / medium / far) from the box’s apparent size. Because apparent size conflates true size with distance, Lumen applies *per-class size priors*: a laptop occupying 40% of the frame width is near, and a cup occupying 18% is equally near, because cups are small. Qualitative spatial representations of this kind have been shown sufficient for indoor assistive guidance, where approximate awareness — not metric depth — drives safe and intuitive instructions [13]. Where metric distance genuinely matters — how many steps to a door — Lumen instead uses a pinhole-camera model over a known object height (Section 5.2.6).

2.1.3 Semantic Segmentation and Monocular Depth

Object detectors can only report objects they have a name for, yet the clutter that actually litters floors — clothes piles, boxes, bins — is largely absent from detection vocabularies. Two class-agnostic perception tools close this gap. *Semantic segmentation* labels every pixel with a scene category; transformer architectures such as SegFormer [14], trained on the ADE20K scene dataset [15], can classify floor, rug, wall, and door pixels in real time. This turns the question “is there an obstacle?” into the more tractable “is the strip of floor I am about to walk on still floor?”. *Monocular depth estimation* models such as Depth Anything V2 [16] predict relative per-pixel depth from a single image, enabling a complementary tripwire signal: a patch of the walking lane that reads much nearer than the floor beside it is an intrusion. Lumen’s obstacle watchdog (Section 5.2.7) uses floor segmentation as its primary class-agnostic signal, with the depth tripwire retained as an alternative.

2.1.4 Hand Pose Estimation

Reach Guidance requires knowing where the user’s hand — specifically the fingertip — is relative to the target object. MediaPipe Hands [17] provides 21 three-dimensional hand landmarks from a monocular camera in real time on CPU, with models compact enough to ship inside the library itself. Lumen uses the index fingertip and wrist landmarks to compute hand-to-target guidance and to detect the moment the fingertip enters the target’s bounding box.

2.2 Speech-to-Text (STT)

Speech-to-Text serves as the primary input modality in voice-driven assistive systems, converting spoken commands into text for downstream interpretation. This enables hands-free interaction, which is essential for users who cannot rely on visual interfaces [18]. A typical STT pipeline consists of audio capture, acoustic feature extraction, and neural transcription [19].

Lumen adopts Whisper [20] as its recognition model, in the CTranslate2-based *faster-whisper* implementation, which runs the same weights several times faster on CPU. Whisper is trained on large-scale, diverse audio, yielding robust transcription across accents, speaking styles, and background noise — the conditions of real households. Robustness matters more than vocabulary here: Lumen’s utterances are short imperative commands, and the transcription errors that do occur (“find my cop” for *cup*, “fone” for *phone*) are handled downstream by the command parser rather than by widening the recognition model [21].

2.3 Language Understanding

After transcription, the text must be mapped to an explicit system action. Large language models offer great flexibility for this mapping [22], but assistive systems demand deterministic, low-latency, and auditable behavior; misinterpreting a command has physical consequences. Lumen therefore uses a constrained command parser built on fuzzy string matching: every possible (prefix, target) split of the utterance is scored against known command templates (“find my ...”, “navigate to the ...”) and known target vocabularies, with synonym and mishearing normalization applied first. This follows the long-standing insight from task-oriented dialog systems that a limited, well-defined intent schema yields reliable and predictable interaction [23], [24]. Completion phrases (“got it”, “stop”) and information queries (“describe”, “repeat that”) are recognized the same way during active tasks.

2.4 Text-to-Speech (TTS)

Text-to-Speech completes the closed interaction loop, delivering guidance in auditory form — the primary output channel for blind users [25]. Modern neural pipelines synthesize natural speech from text in a single architecture [26]; classical parametric synthesis research established the clarity and latency requirements that interactive systems must meet [27]. Lumen uses Google’s gTTS service, which produces consistent, natural MP3 audio [28]. Two engineering measures adapt a cloud TTS service to real-time guidance: synthesized clips are cached in a bounded LRU keyed by exact text (guidance phrases

repeat constantly), and synthesis runs in a background speech queue so that network latency never stalls perception (Section 6.5). Assistive TTS additionally demands *discipline* over *quantity*: Lumen enforces a two-tier speech policy in which priority announcements always play, while ambient nudges are dropped rather than queued whenever the audio pipe is busy.

2.5 Browser Device Sensing

Delivering the client as a web page constrains sensing to what browsers expose — and modern browsers expose enough. `getUserMedia` provides camera frames and microphone audio (requiring HTTPS outside localhost); `MediaRecorder` captures push-to-talk utterances; `DeviceMotionEvent` reports coarse accelerometer activity, which Lumen classifies into still / moving / walking; `DeviceOrientationEvent` reports the compass heading that anchors the navigation room scan; the Vibration API delivers haptic cues; and the Wake Lock API keeps the screen (and thus the camera) alive. iOS Safari imposes additional constraints — sensor permissions must be requested from within a user gesture, and audio playback must be unlocked by a user action — which shaped the client’s startup sequence (Section 6.9).

2.6 State-Based System Control

Finite state machines provide deterministic behavior, clear task boundaries, and predictable recovery — properties long exploited in robotics control architectures [29] and their modern successors such as behavior trees [30]. In Lumen, an explicit FSM is the authority on what the system is doing at every moment: idle, listening for a command, executing a specific task, or returning to idle. Perception runs only inside active task states, which is precisely what implements the task-scoped paradigm; the FSM’s explicit transitions are also what make the system testable, since every behavioral claim can be phrased as “event X in state Y produces state Z and utterance W”. The exploration navigation task nests a second, task-internal state machine (scan, face target, approach door, confirm arrival) beneath the session FSM, following the same principle at a finer grain.

Chapter 3

Related Work

3.1 Assistive Vision Systems for Blind Users

Assistive vision systems for blind and visually impaired users have been an active area of research for several decades, driven by advances in computer vision, wearable sensing, and audio-based interaction. These systems typically employ cameras mounted on wearable or handheld devices to capture visual information from the user’s environment and translate it into non-visual feedback, most commonly through speech or spatialized audio. The primary objective is to enhance environmental awareness, enabling users to perceive obstacles, objects, and spatial layout that would otherwise be inaccessible through traditional mobility aids alone.

Early assistive vision approaches focused on specific perceptual functions such as obstacle detection, depth estimation, and object recognition, often providing simple auditory cues or alerts to indicate the presence of hazards [3]. As computer vision techniques matured, more comprehensive systems emerged that attempted to describe broader aspects of the surrounding scene, including object categories, relative positions, and semantic context [5]. These systems generally rely on continuous or near-continuous visual processing pipelines, transforming camera input into spoken descriptions intended to substitute or augment visual perception.

More recent work has explored increasingly sophisticated capabilities, including semantic scene understanding, indoor layout recognition, and landmark identification [2]. Advances in deep learning have enabled higher recognition accuracy and robustness in complex indoor environments, while improvements in wearable hardware have facilitated hands-free operation through smart glasses and body-mounted cameras. Many of these systems emphasize real-time feedback and aim to provide a rich representation of the user’s surroundings, often prioritizing completeness of information over selectivity.

Despite significant progress, existing assistive vision systems vary widely in how perception is activated, how information is structured, and how feedback is delivered to users.

This diversity reflects differing assumptions about user needs and system goals, and it motivates a closer examination of interaction paradigms, information delivery strategies, and control structures, discussed in the following sections.

3.2 Interaction Paradigms: Continuous Narration vs. On-Demand Assistance

A central design dimension in assistive vision systems is the interaction paradigm that governs when perception is activated and how information is delivered. Prior work largely falls into two dominant paradigms: continuous scene narration and on-demand, query-based assistance.

Continuous narration systems aim to provide persistent situational awareness by continuously processing visual input and verbalizing detected objects, obstacles, and scene elements [2], [5]. While this approach can increase environmental coverage, it often results in verbose and redundant feedback, requiring users to continuously filter relevant information from irrelevant details. Studies in assistive technology and auditory interfaces have shown that such uninterrupted information streams can significantly increase cognitive load, interfere with attention to ambient sounds, and reduce overall task performance [6], [7].

In contrast, on-demand and query-based assistive systems restrict perception and feedback to moments explicitly initiated by the user through individual queries [5], [8]. By limiting feedback to user-triggered queries, on-demand approaches reduce continuous cognitive burden and offer greater control over information flow. However, they introduce a different challenge: users must know what to ask for and when. In unfamiliar environments, blind users may lack sufficient situational context to formulate appropriate queries, leading to delayed or incomplete assistance.

Both paradigms reflect implicit assumptions about user behavior and system responsibility. Continuous narration assumes that more information leads to better awareness; query-based systems assume that users can effectively manage their own information needs. In practice, neither assumption consistently holds. This tension highlights a gap in current assistive vision research: the lack of interaction models that dynamically align perception and feedback with user intent and task context. Lumen’s task-scoped paradigm addresses exactly this gap — perception is activated by a task, structured by that task’s phases, and terminated with it — and its navigation mode goes one step further by treating the *user* as a collaborative resource, asked for a direction hint only when perception alone is genuinely ambiguous.

3.3 Indoor Navigation and Object-Finding Systems

Indoor navigation and object-finding represent two of the most common and challenging assistive tasks. Unlike outdoor navigation, which can rely on GPS and large-scale mapping, indoor environments are highly structured, dynamic, and GPS-denied, requiring alternative sensing and localization strategies [2].

Indoor navigation systems for blind users often guide users between predefined locations using landmarks, visual markers, or spatial cues extracted from the environment [31], [32]. Notably, infrastructure-based systems such as NavCog [33] achieve reliable turn-by-turn guidance by instrumenting the building with beacons and pre-built maps — an approach that works well in flagship venues but does not scale to arbitrary homes and offices. Vision-based approaches avoid instrumentation by detecting doors, corridors, and signage from camera input, but many rely on prior mapping or fiducial markers, and most assume a known route rather than genuine exploration of an unknown space. Lumen’s navigation deliberately occupies the uninstrumented end of this spectrum: no maps, no markers, no SLAM — exploration driven by door detection and semantic room recognition, with the human contributing coarse direction knowledge that the sensors cannot provide.

Object-finding systems help users locate specific items within their immediate surroundings [34], [35], commonly employing real-time object detection and audio cues to guide users toward a target. Although object-finding is often treated as a simpler problem than navigation, it presents similar challenges in perception reliability, spatial feedback, and interaction flow. Many existing systems continuously scan the environment or require repeated user queries, offering limited structure for task initiation, progression, and termination. Lumen structures the entire object-finding interaction as one task with explicit phases — search, track, reach, grasp — ending either by the user’s confirmation or by the system detecting the grasp itself.

3.4 Cognitive Load and Information Management

Cognitive load has been widely recognized as a critical factor in the effectiveness of assistive technologies. Assistive systems rely on auditory feedback as the primary communication channel, requiring users to process spoken information while simultaneously attending to environmental sounds, spatial cues, and physical navigation. Excessive or poorly structured auditory information overwhelms users, reducing task performance and situational awareness [6], [7].

Beyond the quantity of information, prior work emphasizes timing, structure, and context: information presented outside an immediate task context is less likely to be retained and more likely to interfere with concurrent cognitive processes [36]. Despite

these insights, many assistive vision systems treat cognitive load as a secondary concern, addressed through ad hoc verbosity reduction rather than architectural mechanisms that regulate when perception is active and when information is delivered.

Lumen treats cognitive load management as a first-class design objective at three levels: the task-scoped paradigm bounds *when* the system may speak at all; a two-tier speech policy distinguishes priority announcements from droppable ambient nudges, so guidance never queues up and lags behind reality; and utterance merging ensures that no spoken line is ever interrupted by the next one.

3.5 Wearable Platforms and Voice-Based Interaction

Wearable platforms — smart glasses, head- or chest-mounted cameras, belt-worn systems — have been the dominant form factor for assistive vision, as they enable hands-free sensing [3], [35]. Their design is constrained by size, weight, battery life, and computational resources; consequently, many systems offload processing to paired smartphones or cloud services [37], trading local constraints for latency and connectivity challenges.

Voice-based interaction has emerged as the natural control modality for such systems [38], but speech interfaces require careful design to avoid ambiguity, misrecognition, and dialogue overhead, particularly in noisy indoor environments. Moreover, many systems treat voice as a thin control layer mapping commands directly to isolated functions, with limited consideration of interaction flow, task progression, or system state.

Lumen draws a pragmatic conclusion from this literature: the modern smartphone *already is* the wearable — camera, microphone, speaker, IMU, compass, and connectivity in one ubiquitous device — and delivering the client as a web page removes even the app-install barrier. The engineering effort thus shifts from hardware to interaction structure: an explicit state machine governs how tasks are initiated, executed, interrupted, and terminated by voice.

3.6 System Architectures, Control Models, and Safety

The effectiveness of assistive technologies depends not only on sensing accuracy and interaction modalities, but also on the underlying architecture that governs task execution, control flow, and error handling. Prior work commonly describes functional components — perception, localization, feedback — while leaving the coordinating control logic implicit [3], [35]. Many systems adopt reactive architectures where sensory inputs directly trigger responses [38]; without explicit state, perception may remain active indefinitely, feedback may repeat unpredictably, and behavior under edge cases is hard to reason about.

Safety and failure handling are particularly critical for blind users, for whom incorrect guidance has immediate physical consequences, yet most systems rely on ad hoc safeguards such as confidence thresholds [2]. In robotics and human-computer interaction, state-based control models — finite state machines and their descendants — have long provided deterministic behavior, clear task boundaries, and predictable recovery [29], [30], but have seen limited adoption in assistive vision systems.

Lumen adopts explicit state-machine control at both the session level and inside the navigation task, and extends the safety-first philosophy into perception itself: every navigation-critical percept must survive multiple independent checks (temporal consistency, geometric gates, semantic verification) before it is ever spoken to the user, and obstacle warnings preempt all other guidance while the user is walking. Because the decision logic is pure and clock-injected, these safety behaviors are not just design intentions — they are verified by automated tests.

3.7 Summary and Research Gap

Across this literature, three gaps recur. First, interaction paradigms are polarized between continuous narration and reactive querying, with little work on structuring assistance around explicit, bounded tasks. Second, uninstrumented indoor navigation — no maps, no beacons, no SLAM — remains largely unaddressed for blind users, despite being the common case in homes and unfamiliar buildings. Third, control logic and safety handling are typically implicit, making system behavior unpredictable and unverifiable. Lumen’s design responds to all three: task-scoped perception under an explicit FSM, goal-directed exploration navigation that fuses semantic room recognition with door detection and human hints, and a verification-first architecture whose behavioral guarantees are enforced by an automated test suite.

Chapter 4

System Design

4.1 From Proposal to Final Design

The original project proposal envisioned a purpose-built wearable device — camera, microphone, and speaker in a dedicated enclosure — streaming to cloud services. Two realizations during development reshaped this design into its final, stronger form.

First, **the smartphone already is the wearable**. A modern phone integrates every sensor the custom device would have carried — camera, microphone, speaker, accelerometer, magnetometer — in hardware that is mass-produced, familiar to the user, and already in their pocket. Building custom hardware would have reproduced this integration work at prototype quality while adding battery, enclosure, and connectivity problems that contribute nothing to the research questions.

Second, **the browser removes the last installation barrier**. Delivering the client as a plain web page — no app store, no APK, no platform approval — means a user (or an evaluator) points their phone at a URL and the system works. The requirements this imposes (HTTPS for camera access, iOS audio unlock quirks, permission gestures) proved solvable, and are documented in Section 6.9.

The navigation task evolved even more substantially. The proposal described *collaborative waypoint identification*: the user names a sequence of landmarks, and the system walks them landmark to landmark. Early experimentation exposed the flaw: in an unfamiliar building, a blind user cannot name intermediate landmarks — that is precisely the information they lack, and demanding it contradicts the cognitive-load motivation of the whole project. The final design inverts the responsibility: the user names only the *destination*, and the system explores — scanning each room, recognizing arrival semantically, choosing doors, and guiding the user through them. The user contributes what they genuinely have (coarse direction knowledge, confirmation of transit) rather than what they lack. What remains from the proposal is the deeper principle both designs share: perception scoped to an explicit task, under explicit state control.

4.2 Requirements

4.2.1 Functional Requirements

- **FR1** — Voice-initiated tasks: the user starts, completes, and aborts every task by voice (push-to-talk), with UI buttons as fallback.
- **FR2** — Object Allocation: locate a requested object from a curated COCO vocabulary and guide the user to it with direction and distance cues.
- **FR3** — Reach Guidance: when the target is within arm’s reach and a hand is visible, guide the hand to the target and complete the task automatically on touch.
- **FR4** — Navigation: guide the user to a named destination room without maps or infrastructure, recognizing arrival from room-indicator objects and traversing doors as needed.
- **FR5** — Obstacle warnings: while the user walks during navigation, warn about objects and unnamed clutter in the walking lane, preempting all other guidance.
- **FR6** — Information queries: “describe” the current view and “repeat” the last utterance on demand during a task.
- **FR7** — Session continuity: an interrupted connection (dropped Wi-Fi, page reload) resumes the active task on reconnect.

4.2.2 Non-Functional Requirements

- **NFR1** — Zero install: the client is a web page; the only client-side requirement is a modern browser.
- **NFR2** — Responsiveness: guidance must follow scene changes closely enough to feel live (detection at $\sim 3\text{--}5$ FPS; spoken reaction within $\sim 1\text{--}2$ s).
- **NFR3** — Speech discipline: no utterance may be cut off by another; ambient guidance must never queue up and lag behind reality.
- **NFR4** — Trustworthy perception: nothing is spoken to the user unless it survived temporal-consistency and (for doors) semantic verification; false guidance is treated as a safety defect, not a cosmetic one.
- **NFR5** — Testability: all decision logic must be executable without cameras, models, or network, so behavior is verifiable in an automated suite.
- **NFR6** — Cane-complementary: the system provides advance information; it does not replace the white cane’s ground-truth contact zone.

4.3 Architecture Overview

Lumen is a thin-client / smart-server system connected by a single WebSocket (Figure 4.1). The browser client captures and forwards sensor data and plays back audio; it

holds no task state. The backend owns everything else: speech recognition, command parsing, the session state machine, the per-task perception loops, and speech synthesis. Serving the frontend from the same process (same origin) means one tunnel or host exposes the entire system.

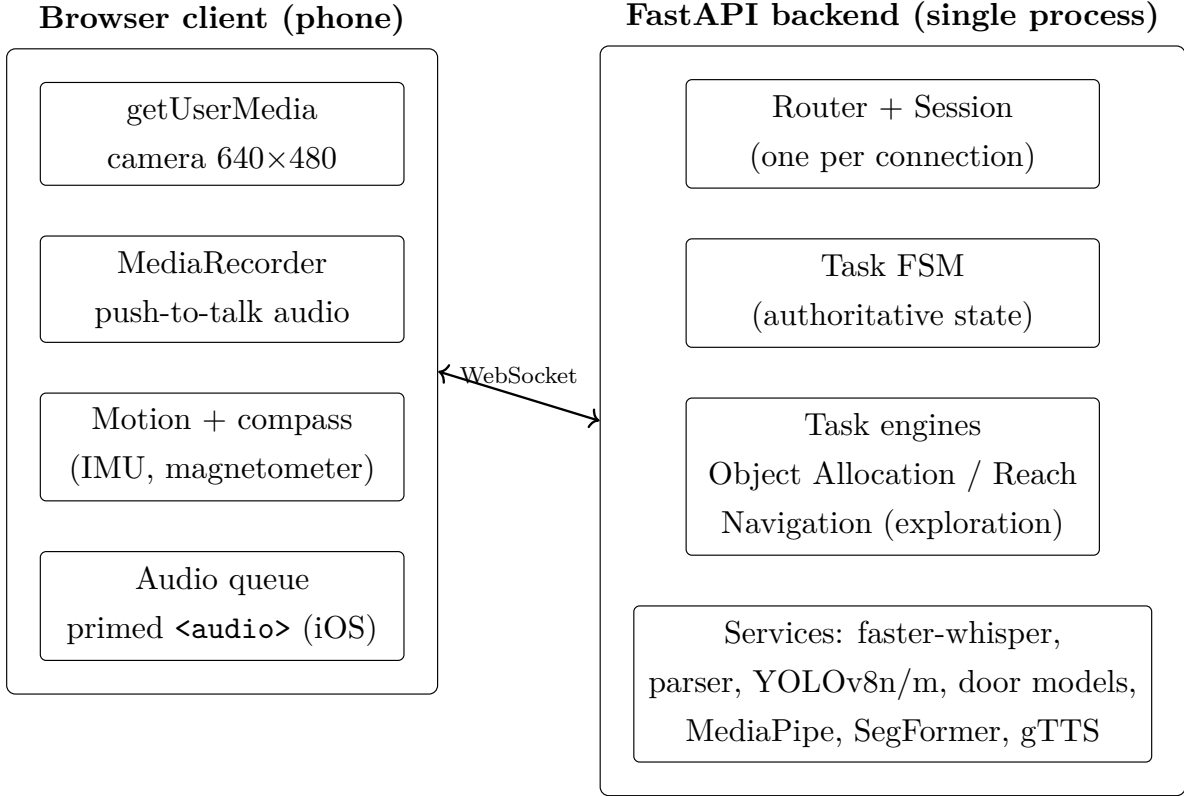


Figure 4.1: System architecture: a sensing-only browser client and a single-process backend joined by one WebSocket carrying JSON control messages and tagged binary media.

4.3.1 Wire Protocol

One WebSocket carries two channels. *Text frames* hold JSON control messages: lifecycle events (`user_event`: start / stop / confirm), device sensing (`motion`: still / moving / walking; `heading`: compass degrees, throttled to $\geq 2^\circ$ change at ≤ 5 Hz), session resumption (`resume`), and, server-to-client, state pushes (`fsm_state`), transcriptions, haptic cues, and errors. *Binary frames* carry media, each prefixed by a one-byte tag: `0x01` JPEG camera frames (client→server, ~ 5 FPS), `0x02` WebM/Opus push-to-talk recordings (client→server), and `0x03` MP3 speech clips (server→client). The protocol is frozen and documented, and unknown messages are logged and ignored, which lets client and server evolve independently.

4.3.2 Session Model

Each WebSocket connection owns a **Session**: the FSM instance, the latest decoded frame and its timestamp, the latest motion state and compass heading, the task context (target object or destination), and a handle to the running task loop. There is no shared task state across users — multi-user isolation is a property of the architecture. A resume pool preserves the active task for five minutes across disconnects: when the phone drops Wi-Fi mid-task and reconnects, the client presents its previous session id and the server re-fires the task with a spoken “Resuming...” announcement (FR7).

4.4 The Task Finite State Machine

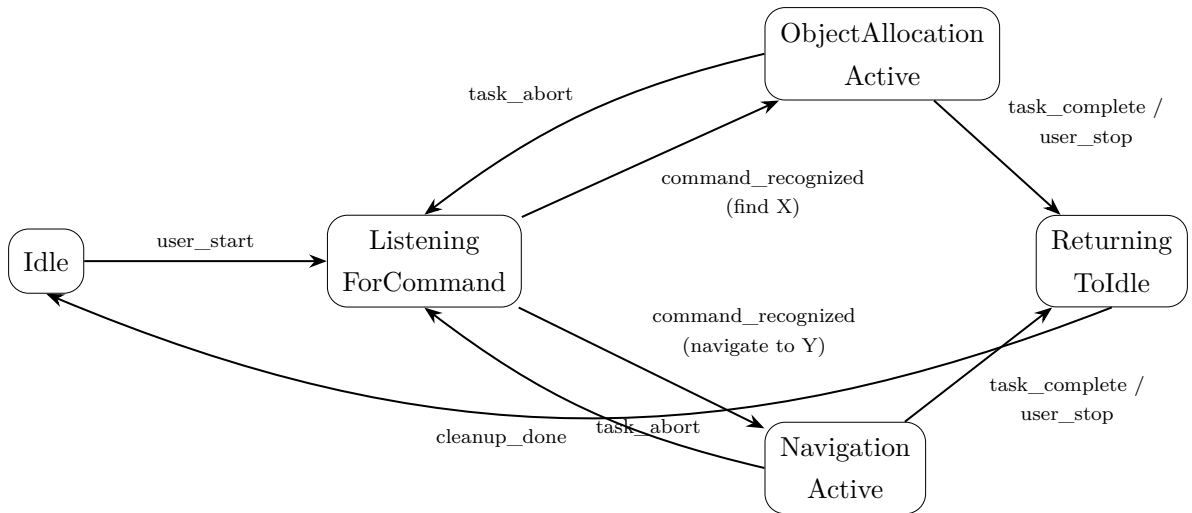


Figure 4.2: The session FSM. Every transition is driven by an explicit event; the only autonomous transitions are `cleanup_done` (one tick after entering `ReturningToIdle`) and `task_complete` fired by a task engine on detected success (grasp, arrival).

The FSM is authoritative: the client renders whatever state the server pushes and maintains no state of its own. Entry and exit hooks on the two active states start and cancel the corresponding task engine’s asynchronous loop, so a task can never outlive its state — cancellation, completion, disconnection, and errors all funnel through the same teardown path. Reach Guidance runs as a phase *inside* `ObjectAllocationActive` rather than as a separate FSM state, because the transition between far-field guidance and hand guidance is a perception decision (target near and hand visible), not a user decision.

4.5 Voice Interaction Design

Push-to-talk, rather than an always-open microphone, keeps the user in control and sidesteps both privacy concerns and false activations. Utterances flow through faster-

whisper, then the fuzzy command parser (Section 2.3), which classifies them as task commands (“find my cup”, “navigate to the kitchen”), completions (“got it” / “stop”), or information queries (“describe”, “repeat that”). The parser’s navigation vocabulary is deliberately kept in lockstep with the set of rooms the navigation engine can actually recognize — a destination the parser accepts but the explorer cannot find would produce two contradictory spoken responses.

Every accepted command is confirmed by voice (“Looking for your cup.”), every rejection prompts a retry (“I didn’t catch that, please repeat.”), and every task end is announced. Haptic cues complement voice at key moments — entering reach range, touching the target, task completion — degrading silently on devices without a vibration API.

4.6 Safety Design

Four principles govern safety-relevant behavior:

1. **Never speak unverified perception.** Objects require temporal consistency (3-of-5 frames in Object Allocation; localized sighting counts in Navigation); doors additionally pass geometric gates and semantic verification (Section 5.2.3).
2. **Obstacles preempt everything.** While the user is walking, the obstacle watchdog’s warnings override door guidance until the path clears.
3. **The camera is the odometer.** “You walked through the door” is only ever inferred when sustained camera motion accompanies the visual evidence — detection flicker alone can never fake progress.
4. **Bounded autonomy.** Exploration checks in with the user after a configurable number of rooms (“say stop anytime...”), and a whole-journey timeout guarantees the system never wanders indefinitely; “stop” is honored from any state at any time.

Lumen is designed as a complement to the white cane: the cane owns the contact zone (~ 1 m); Lumen’s value is the 2–4 m advance information — where the door is, what is in the path, which room this is. This division of labor is communicated to users and maintained throughout the guidance phrasing, which directs attention and movement but never promises collision safety.

Chapter 5

Task Families and Algorithms

The system organizes all assistance into **two task families**: *Object Allocation*, which finds a requested object and guides the user all the way to grasping it, and *Navigation*, which guides the user to a destination room through goal-directed exploration. Every capability the user can invoke resolves into one of these two: hand-relative *reach guidance* is the terminal phase of an Object Allocation task, and the *information queries* (“describe”, “repeat”) are auxiliary services built on the Object Allocation perception stack. This chapter details both families as implemented. Throughout, a common structure recurs: a perception stage that turns frames into *trusted* detections, a pure decision layer that turns detections into guidance, and a speech policy that decides what actually reaches the user’s ears. The perception stages differ per family; the philosophy — nothing unverified is ever spoken — does not.

5.1 Object Allocation

Object Allocation answers “find my cup”. When the session FSM enters ObjectAllocation-Active, an asynchronous loop starts, running at 5FPS over the session’s frame stream until the task completes, is cancelled, or times out. The task progresses through two guidance regimes governed by a single tracker: *body-relative* guidance while the object is beyond reach (direction and distance cues that move the user), and *hand-relative* reach guidance once the object is at arm’s length and the user’s hand is visible (cues that move the hand). The hand-off between the two is automatic and reversible.

5.1.1 Detection and Temporal Filtering

Each frame passes through YOLOv8n with inference restricted to the requested COCO class and a 0.35 confidence floor. Raw per-frame detections are too flickery to speak: a *temporal-consistency filter* over a sliding window of the last five frames requires the target to be present in at least three of them before it is trusted, killing single-frame false

positives. When multiple instances are visible (two chairs), guidance targets the most head-on one — the box whose horizontal centre is closest to the frame centre.

5.1.2 Spatial Reasoning with Per-Class Size Priors

The trusted box is classified into a horizontal region — left / center / right, split at 0.35 and 0.65 of frame width — and a distance bucket. Distance uses the larger of the box’s width and height fractions (so both wide and tall objects are handled), compared against a *per-class near-at-arm’s-reach threshold*: a laptop at 40% of frame width is near, a cup at 18% is equally near, a phone at 12% likewise — same arithmetic, three correct answers, because the priors encode real object sizes. The ratio of observed apparent size to the class threshold determines the bucket: at or above 0.85 of the threshold the object is *near*; at or below 0.35 it is *far*; otherwise *medium*. Classes without an explicit prior fall back to a book-sized default. (The complete prior table is given in Chapter 6, Table 6.6.)

Bucket transitions are debounced — two consecutive frames of the same (region, distance) pair are required before the “official” bucket switches — so YOLO’s box jitter at a threshold boundary cannot fire three contradictory phrases in under a second. The debounce exists to filter jitter while the user is still, not to hide real movement: when the client’s motion monitor reports that the user has started moving, the smoothed state is reset and new buckets are accepted immediately, so guidance keeps up with actual position changes.

5.1.3 Guidance Policy

The tracker speaks on first sighting (“Found your cup, to your left”), on bucket changes (“Your cup is straight ahead, a few steps away”), and re-affirms unchanged guidance at most every 6 s. Edge cases have explicit behaviors:

- **Never seen yet:** a periodic scan prompt every 8 s — “I don’t see your cup yet. Slowly turn around so I can look.” The first prompt is delayed by a full interval so it does not talk over the “Looking for your cup” confirmation at task start.
- **Seen, then lost** for a full five-frame window: a one-shot loss announcement including the last known direction — “I lost sight of your cup. It was to your left.”
- **Never seen within 60 s:** the task aborts with “I couldn’t find your cup. Stopping the search.”

Completion is user-driven — the user says “got it” and the task ends with a closing phrase naming the object — with one exception, produced by the reach phase described next.

5.1.4 The Reach Guidance Phase

When the target’s smoothed distance bucket is *near* and MediaPipe detects a hand in frame, guidance switches from body-relative to *hand-relative* cues. MediaPipe is only invoked once the tracker classifies the target as near — running it earlier would waste roughly 15 ms per frame while the target is still across the room.

The controlling quantity is the offset between the index fingertip and the target box centroid. Three states result:

- *approach* — directional cues along the dominant axis: “Move your hand to the right”, “Move your hand up”. The dominant axis is chosen by *pixel* magnitude rather than frame-fraction, because the frame is wider than it is tall and fractional comparison systematically biased cues toward the vertical axis.
- *almost* — fingertip within 15% of the frame from the target centroid on both axes: “Almost there. Reach forward,” accompanied by a short haptic pulse on entry into this state.
- *touching* — fingertip inside the target’s bounding box: “Your hand is on the cup. Grasp it,” with a long haptic pulse.

Touching is the single autonomous success exit in the whole system — it fires the `task_complete` event without user confirmation, because the fingertip on the object *is* the confirmation, and it is emitted at most once per task. Reach cues are throttled: an unchanged (state, direction) pair is re-spoken at most every 5 s. A 3 s grace period keeps the system silent (rather than reverting to body-direction guidance) across momentary hand-detection dropouts, preventing the two guidance regimes from alternating at 5 FPS on hand-detection flicker; leaving reach mode clears the throttle key so re-entering later is announced fresh.

5.1.5 Information Queries

Two voice queries, built directly on the Object Allocation perception stack, are available at any time and never change task state. **“Describe”** (“what do you see”, “what’s in front of me”) runs YOLOv8n over the current frame at a raised 0.5 confidence floor — only things the system is reasonably sure about are worth mentioning to a user with no visual cross-check — and reads back up to the four highest-confidence detections with their regions, classified by the same spatial-reasoning module the search loop uses: “I see a laptop straight ahead, a cup on your left, and a chair on your right.” **“Repeat”** (“say that again”) replays the last spoken guidance phrase, which the session records on every utterance; the replay itself does not overwrite the record, so repeating twice replays the same phrase.

5.2 Navigation: Goal-Directed Exploration

Navigation is the deeper task family. The user names only a destination room; the system explores toward it with no map, no markers, and no localization — the design bet is that *semantic room recognition* plus *door detection* plus the user’s own physical turning, tracked by compass, can substitute for the SLAM a phone camera cannot do. The navigation loop runs at 3 FPS — its models are heavier than Object Allocation’s — and the first navigation task in a server’s lifetime loads them with a spoken “Give me a moment while I get ready” rather than going silent.

5.2.1 Destination Model and Semantic Arrival

Each supported destination maps to *indicator objects*, split into primary (near-unique to the room) and secondary (supporting, often shared across rooms), as shown in Table 5.1. Arrival is declared when at least one primary or at least two secondary indicators are confirmed — so a lone sink never declares a kitchen (bathrooms have sinks), but a refrigerator does. All indicators are COCO classes, so the stock detector produces them with no additional training. Spoken destinations resolve through an alias table (“washroom”, “restroom”, “loo” → bathroom; “lounge”, “den” → living room; ...) with a longest-alias substring fallback, so “take me to the master bedroom” still resolves to *bedroom*; a destination that resolves to nothing ends the task with a spoken list of the rooms the system does know.

Table 5.1: Destination rooms and their indicator objects (all COCO classes). Arrival requires ≥ 1 primary or ≥ 2 secondary indicators, temporally confirmed.

Goal	Primary indicators	Secondary indicators
kitchen	refrigerator, oven, microwave, toaster	sink, dining table
bathroom	toilet	sink
bedroom	bed	—
living room	couch, TV	potted plant, remote
office	laptop, keyboard	mouse, TV
dining room	dining table	chair

Indicator evidence is deliberately *localized*: during a room scan, a class is confirmed only when at least four sightings fall within one 90° window (a 30° sector plus its two neighbours). A real fridge racks up its sightings in one spot as the camera sweeps past; detector noise scatters around the room and can never sum to an arrival.

5.2.2 The Door Detector

COCO has no door class, so a dedicated detector was trained. Two public datasets were merged and reduced to a single **door** class: DoorDetect [39] (386 door images plus 827 hard negatives — handles and cabinets — that teach the model what a door is *not*) and DeepDoors2 [40] (3,000 images with 3,106 door boxes, converted from segmentation masks by thresholding the mask colour and taking connected-component bounding boxes), for a combined 4,213 images split 80/20 into 3,371 training and 842 validation images. A YOLOv8n was fine-tuned from COCO-pretrained weights (319 of 355 layers transferred) at 640 input resolution, batch 8, AdamW; the best checkpoint (epoch 39) achieves **mAP@50** = 0.946, mAP@50–95 = 0.829, precision 0.968, and recall 0.892 — comfortably above the ≥ 0.6 mAP@50 target set for the milestone. A second, 4-class YOLOv8s model (door / handle / cabinet door / refrigerator door) was trained on the original DoorDetect labels; its recall is too low to detect doors alone, but that is not its job — it serves as a *semantic verifier*, below. Both trained weight files are committed to the repository so a fresh clone runs the complete system.

5.2.3 The Three-Layer Door Perception Funnel

A false door is the most dangerous percept in the system — it points a blind user at a wall, a curtain, or a refrigerator. Raw detector confidence cannot prevent this: in live testing, blank walls scored as doors at 0.9 confidence. Every door candidate above the 0.45 confidence floor therefore descends a three-layer funnel (Figure 5.1), each layer motivated by a specific observed failure.

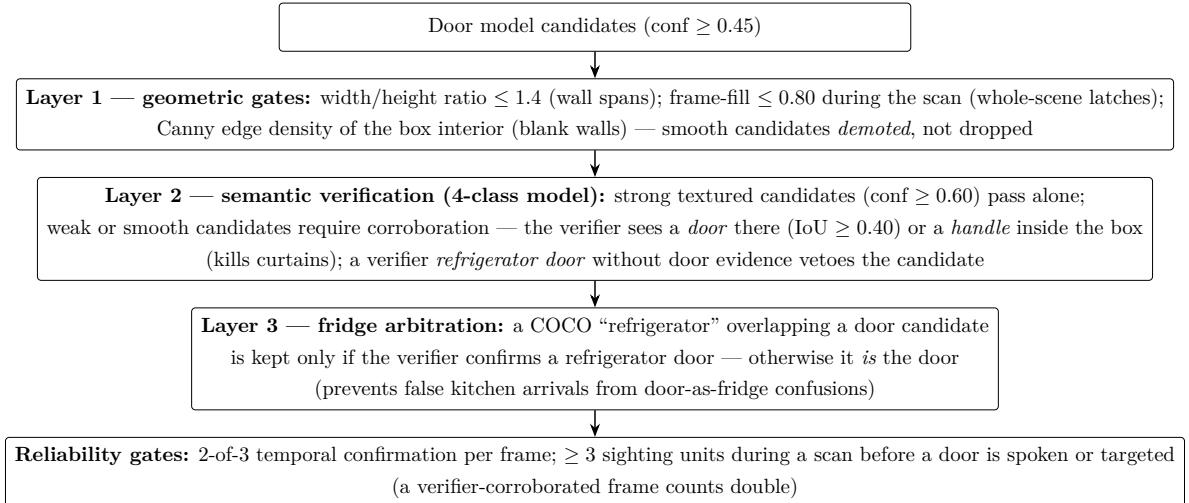


Figure 5.1: The door perception funnel. Each layer was added in response to a specific live failure: whole-wall detections, dim plain doors indistinguishable from walls, a lace curtain that beat every geometric gate, and doors misread as refrigerators.

Several subtleties deserve emphasis. First, edge density is *not* a hard reject. A

plain door in dim light is as smooth as a wall (both around 0.01 edge fraction), and confidence cannot break the tie (walls reach 0.9). Smooth candidates are instead demoted to require semantic proof — only the verifier seeing a door or a handle there can pass them. This is what finally separated the lace curtain (edge-rich, tall, door-sized, defeats all geometry) from real doors: curtains have no handles. Second, the verifier’s fridge veto applies only when it saw fridge evidence *without* door evidence at the same spot — the verifier hallucinates “refrigerator door” on real doors too, and vetoing frames it simultaneously corroborated kept breaking detection streaks on the actual door. Third, a vetoed door never becomes fridge evidence: the verifier also hallucinates “refrigerator door” on blank walls, and feeding those into arrival evidence produced fake fridges; real fridges must be detected by the COCO model on its own. Fourth, fridge arbitration is suspended during the directed indicator confirmation phase, when the user is deliberately pointing the camera at an appliance and the door model routinely fires on real fridges — without the exemption, the arbitration starved the confirmation of the very fridge sightings it was checking for. Finally, where a surviving door box and a COCO object box still overlap heavily ($\text{IoU} \geq 0.5$), only the higher-confidence claim is kept, eliminating door/refrigerator double-claims.

The whole-frame gate is applied to COCO objects too: while the user stands mid-room scanning or confirming, a “refrigerator” boxed edge-to-edge is a wall latch and a fake kitchen indicator — a direct path to a false arrival — so whole-scene object boxes are rejected in those phases. During the close door approach, where filling the frame is legitimate, the gate is lifted.

5.2.4 The Compass-Anchored Room Scan

On entering a room (and at journey start), the system requests one slow full turn: “Looking for the kitchen. Let’s scan the room — slowly turn to your right, all the way around, until you are facing where you started.” The goal echo is folded into this instruction rather than spoken separately, so it cannot be cut off. The first compass heading is locked as the anchor — “ahead” means where the scan began — and every subsequent detection is filed by true bearing: $\text{bearing} = \text{camera heading} + (x_{\text{frame}} - 0.5) \cdot \text{FOV}$, the in-frame correction fixing a class of “said left, was right” errors. Sightings are accumulated into $12 \times 30^\circ$ sectors, and door and indicator bearings are *clustered* (sightings within 30° merge, including across the $\pm 180^\circ$ seam) so one physical door is announced once, with one precise heading. Indicator sightings record their class alongside the bearing, so a later prompt can name what was seen (“I noticed a fridge on your left”), not just point vaguely.

Scan completion requires *both* $\sim 350^\circ$ of accumulated rotation *and* a compass-confirmed return to within 25° of the start heading. An earlier design that ended on sector coverage

alone could be faked by compass noise mid-turn, ending the scan while the user faced the wrong way — and every spoken direction would then be computed from a broken premise. With the current rule, the directions in the scan summary (“I found a door behind you, to the right”) are true at the moment they are heard; if the rotation total is complete but the user has not yet returned to the start heading, the system nudges (“Almost done — keep turning until you face where you started”). During the turn, exactly two gentle progress nudges are spoken — at 90° and 270° of rotation — with deliberately different wordings so ambient de-duplication does not swallow the second.

A motion gate skips detection on blurred frames while the phone pans too fast (frame-difference above a tuned threshold), coaching “Slow down. Move the phone slowly” only after three consecutive fast frames — a single blurred frame is an autofocus hunt, not the user rushing — while still advancing the rotation total so a fast sweep cannot stall completion. Without a compass (a laptop, or permission denied), the scan falls back to a single steady-pan pass over a fixed number of usable frames, with no bearing summaries.

5.2.5 The Exploration Controller

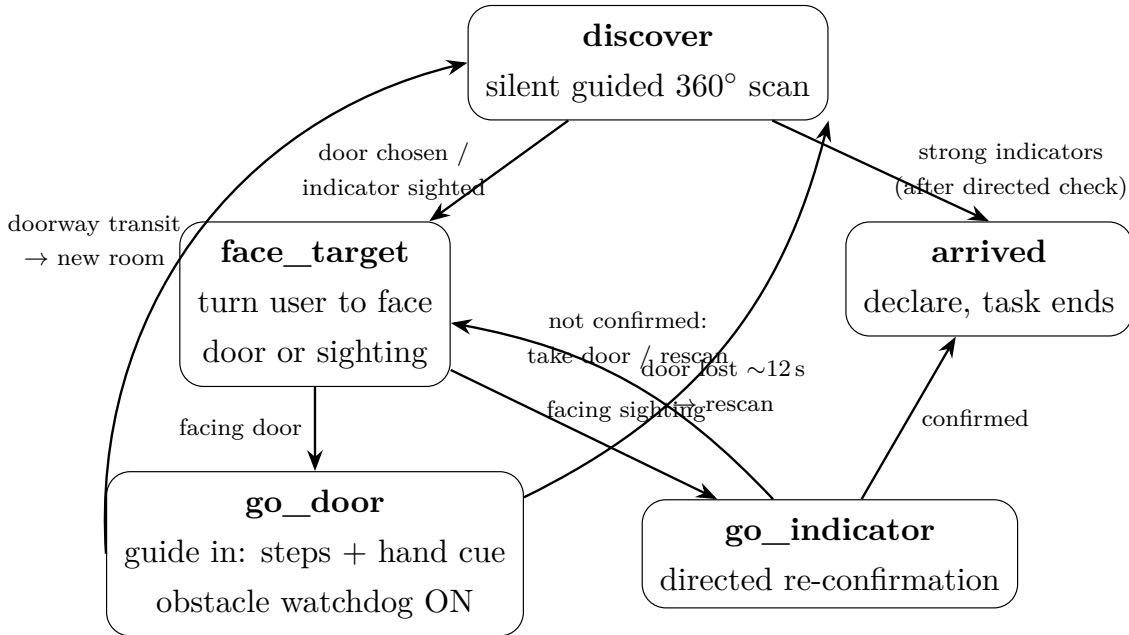


Figure 5.2: The navigation controller’s internal state machine (nested inside the session FSM’s NavigationActive state). Priority is absolute: confirmed indicators always beat doors — the system never walks the user out of the room they were looking for.

The controller (Figure 5.2) implements a two-phase principle: Phase 1 scans silently and collects evidence; Phase 2 acts on that evidence by priority. At scan end, the branches are evaluated strictly in order:

1. **Strong indicator evidence** (the localized arrival bar was met) still receives a *directed double-check*: “I’ve seen signs of the kitchen on your right. Turn that

way, and let’s make sure we’ve reached it.” The user is rotated toward the densest sighting cluster and the system re-confirms there before declaring arrival, so a scan artifact alone never ends a journey. (Without usable bearings — no compass — the confirmation is undirected: “keep panning slowly.”)

2. **Weak indicator sightings** (glimpsed, but below the arrival bar) get the same directed confirmation, naming the most-sighted class in the cluster: “I noticed a fridge on your left. Let’s double-check — turn that way and point the camera at it.” Indicator priority holds: this branch runs before doors are considered.
3. **Doors:** a single confirmed door cluster becomes the target directly: the user is rotated to face it, then guided in. When several clusters survive the sighting minimum, the multi-door dialogue of Section 5.2.9 chooses among them — by the user’s spoken hint when one is given, by proximity straight ahead otherwise. Clusters below the sighting minimum are flickers, never targets. If the scan summary named exactly one finding — this door — the door is not announced twice.
4. **Nothing useful:** the room is rescanned, with the announcement and the fresh scan instruction merged into a single utterance so instructions never collide.

The directed confirmation (`go_indicator`) uses a lower evidence bar than the scan — three sightings per class rather than four — because it is a re-check of evidence the scan already flagged, followed by a short settle window so that neighbouring indicators (the fridge right next to the oven) cross the bar too and the arrival line names *all* of them: “I can see a fridge and an oven — we’ve reached the kitchen.” If confirmation fails within its window (~ 10 s), the indicator had its chance: if the scan also flagged a door, the system takes it (“I couldn’t confirm the kitchen here. Let’s take the door instead”); otherwise a full rescan begins.

The guided turn (`face_target`) computes the signed turn from the live compass heading to the target’s stored absolute heading. The scan-end announcement already said which way to turn, so the turn itself is silent — a re-instruction in different words reads as two contradictory commands — with a sparing nudge (“Turn slowly to your right to face the door”) only if the user has still not faced the target after several seconds. On facing it (within 25°), the handoff is silent in both branches: the very next line is the one-shot door call-out or the arrival itself, and a filler sentence would still be playing when it arrives.

In `go_door`, if the door cannot be re-confirmed for ~ 12 s of observed frames, the flag is treated as stale and a full rescan begins (“I can’t find that door anymore...”), with rotating ambient nudges (“Still looking for a door. Keep moving the camera around the room.”) in the meantime.

5.2.6 Door Approach: Distance in Steps

Door distance uses a pinhole model over the door’s known height: with doors ~ 2 m tall and the focal length derived from an assumed 60° field of view, $d = h_{\text{real}} \cdot f_{\text{px}}/h_{\text{px}}$. Two corrections came from live testing: the focal length must be derived from the image’s *long* side (a camera’s quoted field of view belongs to its wider axis, and phones stream portrait; deriving it from the width understated every distance by $\sim 33\%$), and when the verifier also boxed the door, distance is measured on the tighter (shorter) of the two boxes — the primary model’s boxes run loose at range, and a fat box reads as near. The per-frame estimate is steadied by taking the median over the temporal confirmation window, and a one-shot calibration constant scales the whole chain from a single ground-truth measurement.

Distances are spoken in steps (~ 0.75 m each) with a tactile endgame: within three steps, “Walk about 2 steps, and reach out with your hand to find it”; farther out, “Walk forward, and after about N steps reach out with your hand.” The spoken sequence is strictly ordered: (1) one static call-out stating where the door is — “The door is on your right, about 4 steps away. Let me check the path ahead.” — phrased definitely (“The door is”, never “There’s a door”, which sounds like a guess, because by this point the door has been confirmed); (2) obstacle speech is held for ~ 2 s so that sentence finishes playing; (3) the path verdict follows — either “The path is clear,” plus the walking instruction, or the obstacle warning. Safety information always precedes the instruction to move.

5.2.7 The Obstacle Watchdog

Armed only in the walking phase (`go_door`) — the only phase the user moves — the watchdog fuses two layers with absolute speech priority over door guidance.

Layer A (named objects). During the approach, the COCO detector additionally requests a set of obstacle classes (person, chair, couch, bed, table, plant, bags, bench, TV, dog, cat). Any box that sits low in frame (bottom below 62% of frame height — on the floor ahead, not hanging in the distance), is large enough to matter (at least 18% of frame height), and overlaps the central walking lane (the 30–70% horizontal band) by at least 10% of frame width triggers a warning naming the object: “There’s a chair in your path. Step to your right, where it’s clear,” with the sidestep direction chosen away from the obstacle’s side of the lane. The minimum-overlap threshold exists because side furniture merely grazing the lane edge used to trigger stop warnings.

Layer B (unnamed clutter). COCO has no word for the clutter that actually litters floors — clothes piles, boxes, bins, a standing fan. The question was never “is there an object?” but “is the strip of floor I am about to walk on clear?” — so the class-agnostic signal is *floor segmentation*: SegFormer labels every pixel, and the lane just ahead of the user’s feet (rows 66–94% of frame height, the central 38–62% band)

must remain at least 60% walkable — floor, rug, or carpet pixels, plus *door*, since in this phase the door is the destination, never an obstacle. Something *beside* the path leaves the floor corridor continuous (silent); something *in* the path punches a hole in it (warn, as “something”, with the sidestep direction toward the flank showing more visible floor). This floor-based formulation replaced an earlier relative-depth tripwire [16] that could not reliably separate beside-path from in-path objects in cluttered rooms; the depth variant is retained in the code as a configurable alternative, and if neither auxiliary model is available the watchdog degrades gracefully to the named-object layer alone.

Both layers are debounced — three consecutive blocked frames before the first warning, two clear frames before release — re-warn every ~ 3 s while blocked (“It’s still in your path — step more to your right”), and close with “Okay, the way ahead is clear,” followed, in the same utterance, by re-orientation toward the door if it is in sight (two back-to-back priority lines would cut each other off). While an obstacle episode is open, a blocking frame suppresses door guidance entirely. Once the user is at or through the doorway, the floor signal is meaningless (the panel fills the frame) and pauses — but a person stepping into the doorway is still a named YOLO box, so the named layer stays armed through the transit.

5.2.8 Doorway Transit Detection

Detecting “the user just walked through” has no single reliable signal, so the system fuses several, each gated by the others. A tracked approach must first get *near* the door: either the median metric distance dropped below 2 m, or the confirmed door grew to fill at least 80% of the frame height — a scale-free signal immune to calibration error. Then a saturated door-shaped box at arm’s length — which the edge gate deliberately rejects as a confirmed door, since a panel up close is smooth — arms an “at the door” latch, but only after two consecutive qualifying frames (a single loose box mid-sidestep once spoke “you’re at the door” from two meters away) and never while an obstacle episode is open (the episode must close with its all-clear first, or the spoken order contradicts itself). Arming the latch speaks the whole door sequence in one utterance — network latency between separate commands would leave the user waiting at the door: “You’re right at the door. Reach out with your hand, open it, walk through the doorway, and take two or three steps into the room.”

Transit is then inferred when the door disappears from view for ~ 2 s of observed frames *and* the camera reports sustained motion — at least four frames of frame-to-frame movement since reaching the door. The camera is the odometer: walking produces sustained frame motion, standing still reads near zero, and detection flicker without movement can never fake a walk-through; if the visual thresholds fire but the user has not moved, the at-the-door instruction is gently repeated instead. Far doors glimpsed mid-

walk-through belong to the *next* room and deliberately do not reset the disappearance counter — letting them reset it once stalled the “you’re through” announcement by ~ 16 s. Finally, a hold cap (~ 13 s) forces the inference if a new room’s own saturated door candidates keep the latch alive after the user has clearly moved on. On transit, the new room’s scan begins immediately, anchored to the user’s new heading, with the announcement and the scan instruction merged: “You’re through. Now slowly turn to your right, all the way around...”

5.2.9 Multi-Door Rooms

When a scan finds several door clusters, the choice is conversational before it is autonomous. The scan summary has already told the user how many doors there are and where each one lies (“I found two doors: one ahead, and one on your left”), so Lumen follows it with a question rather than a silent decision: if the user knows which way the goal lies, they are invited to say so. A one-word answer — “left”, “right”, “ahead”, “behind” — is matched against the doors’ bearings, the corresponding cluster becomes the target, and guidance proceeds toward it. The user is treated as a *direction oracle*: someone in their own home often knows the way even when they cannot see it, and a spoken word is cheaper than any amount of additional perception. If the user does not know — they say so, or simply give no hint — the system decides for them, taking the door nearest straight ahead: the choice that requires the least turning, keeps the camera’s evidence for it freshest, and is simplest to narrate. Either way the user is never stranded on an unanswered question, every door was already named by direction so the alternatives are known, and “stop” is honored at any moment. Remembering the door just entered through — so the same fork never has to be asked about twice — is a designed extension discussed in Chapter 9.

5.2.10 Speech Discipline

A blind user cannot glance at a transcript; overlapping or stale audio is a functional bug, not a cosmetic one. The navigation engine enforces a two-tier policy. *Priority* lines (scan summaries, phase changes, obstacle warnings, the at-door sequence, arrival) are always spoken. *Ambient* nudges (keep scanning, turn corrections, door-hunt prompts) are dropped — never queued — if an identical ambient line played within the last 8 s or if *any* line played within the last 2.5 s: the client plays clips back-to-back, and unthrottled ambient chatter would queue up and lag behind reality. Announcements that would otherwise be adjacent are merged into single utterances, because two back-to-back priority lines would cut each other off; a flag carried across the phase change suppresses the next phase’s opening instruction when the previous announcement already contained it. The scan-summary sentence exemplifies all three rules: completion confirmation, findings by

direction, and the next instruction arrive as one connected utterance — “You’re back where you started — scan complete. I found a door behind you, to the right. Turn toward it and point your camera at it, so I can guide you in precisely.”

Chapter 6

Implementation

This chapter documents the system as it exists in the repository: the technology choices, the wire protocol, the backend’s layered organization, the two task engines, the frontend, deployment, and the test infrastructure. Wherever a number appears — a threshold, a rate, a file size, a line count — it is the value in the committed source.

6.1 Technology Stack

Table 6.1 summarizes the technology choices; each was made for a concrete engineering reason rather than familiarity.

Python dependencies are version-pinned in `requirements.txt`, with two deliberate exceptions documented inline: `ultralytics` and `faster-whisper` carry open lower bounds so `pip` resolves Python-3.13-compatible builds of their transitive dependencies (`torch`, `torchvision`, `PyAV`). `faster-whisper` was chosen over the reference `openai-whisper` package specifically because the latter’s installer breaks on Python 3.13 (a `locals()`-after-`exec()` pattern invalidated by PEP 667), while `faster-whisper` ships prebuilt wheels, runs the same model weights $\sim 4\times$ faster on CPU via `CTranslate2`, and pulls in no deep-learning framework of its own.

6.2 Code Organization at a Glance

The repository holds the backend, the frontend, the door-model training pipeline, and the design documents. Table 6.2 gives the implementation’s scale; the counts are of the committed source.

Table 6.1: Technology choices and their rationale.

Layer	Choice	Why
Server	FastAPI + uvicorn	Async WebSockets; a single process serves the API, the WebSocket, and the frontend same-origin.
STT	faster-whisper (base)	Whisper weights on a CTranslate2 backend: $\sim 4\times$ faster CPU inference, pre-built Python 3.13 wheels, no heavyweight framework dependency.
Audio decode	PyAV	Bundles the FFmpeg shared libraries inside the wheel — no system <code>ffmpeg</code> required to decode push-to-talk audio.
Detection (objects)	YOLOv8n	~ 6 MB, CPU-real-time at 5 FPS, COCO classes match the task vocabulary verbatim.
Detection (navigation)	YOLOv8m + 2 custom door models	Recall on room indicators at angle and distance is the bottleneck, not inference time; the door models fill COCO’s missing class.
Floor segmentation	SegFormer-B0 (ADE20K)	Per-pixel walkability for the obstacle lane; optional — degrades to named objects only.
Hand pose	MediaPipe Hands	21 landmarks, CPU real-time; the model file is auto-downloaded and cached like the YOLO weights.
TTS	gTTS + LRU cache	Natural voice; a bounded 50-entry cache keyed by exact text absorbs the repeated guidance templates.
Command parsing	rapidfuzz	Structural prefix-plus-noun fuzzy matching tolerates Whisper mishearings.
Frontend	Vanilla HTML/JS	No build step; seven small ES modules, served by the backend itself.

Table 6.2: Implementation scale by component (lines of committed source).

Component	Lines	Contents
Backend (<code>backend/</code> , excl. tests)	5,403	API layer, FSM, all services, both task engines
of which <code>services/navigation/</code>	2,190	the nine-module navigation package
Automated tests (<code>backend/tests/</code>)	2,079	13 test modules, 281 collected cases
Frontend (<code>frontend/</code>)	1,435	7 ES modules + page + stylesheet
Door training (<code>door_training/</code>)	~ 400	dataset converters, builder, device check

6.3 The Wire Protocol

A single WebSocket per session carries two channels: JSON text frames for control, and tagged binary frames for media. Every binary frame is prefixed with a one-byte type tag (Table 6.3); every JSON message carries a `type` field (Table 6.4). The contract was frozen early in development and is documented in `docs/protocol.md`; anything out of protocol is logged and ignored, optionally answered with a `protocol_violation` error, so an old client can never crash the server.

Table 6.3: Binary frame tags on the shared WebSocket.

Tag	Direction	Payload
0x01	client $\rightarrow$ server	one JPEG camera frame (640×480 , quality 70, ~ 5 FPS)
0x02	client $\rightarrow$ server	one WebM/Opus push-to-talk recording (on button release)
0x03	server $\rightarrow$ client	one MP3 TTS clip (queued and played sequentially)

Table 6.4: JSON control messages on the shared WebSocket.

Message	Direction	Purpose
<code>user_event</code>	$c \rightarrow s$	UI lifecycle: <code>start</code> , <code>stop</code> , <code>cancel</code> , <code>confirm</code>
<code>motion</code>	$c \rightarrow s$	accelerometer state (<code>still</code> / <code>moving</code> / <code>walking</code>), sent on change only
<code>heading</code>	$c \rightarrow s$	compass heading in degrees, throttled to $\geq 2^\circ$ change at ≤ 5 Hz
<code>resume</code>	$c \rightarrow s$	previous session id, to restore a task after a dropped connection
<code>session_hello</code>	$s \rightarrow c$	the new session's id, stored client-side for later resume
<code>fsm_state</code>	$s \rightarrow c$	authoritative task state on every FSM transition
<code>transcription</code>	$s \rightarrow c$	Whisper text + confidence proxy, for visual audit
<code>haptic</code>	$s \rightarrow c$	symbolic vibration pattern name (Section 6.8)
<code>error</code>	$s \rightarrow c$	non-fatal, machine-readable error with code and message

6.4 Backend Organization

The backend is a single Python process organized in four layers.

API layer (`api/`). The WebSocket router dispatches incoming messages: JSON text frames by their `type` field, binary frames by their one-byte tag. The frame handler decodes

JPEG payloads to RGB arrays via Pillow and stores the latest frame (with its timestamp) on the session — task loops always read the freshest frame rather than queuing frames, so processing can never fall progressively behind the camera. The audio handler runs the complete push-to-talk pipeline described in Section 6.5. The `Session` object owns all per-connection state: the FSM, the latest decoded frame and its timestamp, the latest motion state and compass heading, the task context (target or destination, start time), the last spoken phrase (for the “repeat” command), and the handle to the running task loop’s `asyncio` task; an `asyncio.Lock` serializes concurrent sends over the socket, since the `WebSocket` object is not safe under overlapping `send` calls.

FSM layer (`fsm/`). The five-state task machine of Section 4.4, implemented as an explicit transition function with subscriber callbacks; Table 6.5 lists the complete transition table as implemented — every pair not shown is rejected with a logged warning and no state change. The session subscribes: on every transition it runs exit hooks for the old state (cancelling the old task loop), entry hooks for the new state (starting the new task loop, reading the target from the task context), and pushes the new wire-format state name to the client. Entering the transitional `ReturningToIdle` state schedules `cleanup_done` for the next event-loop tick, so the client observes both transitions in order.

Table 6.5: The complete FSM transition table as implemented. Events not listed for a state are rejected (logged, no transition).

Current state	Event	Next state
Idle	<code>user_start</code>	Listening
Listening	<code>command_recognized</code> (object)	ObjectAllocationActive
Listening	<code>command_recognized</code> (navigation)	NavigationActive
Listening	<code>command_unknown</code>	Listening (no-op)
ObjectAllocation	<code>task_complete</code>	ReturningToIdle
Navigation	<code>task_abort</code>	Listening
ObjectAllocation	<code>task_abort</code>	Listening
Navigation	<code>task_abort</code>	Listening
any non-Idle state	<code>user_stop</code>	ReturningToIdle
ReturningToIdle	<code>cleanup_done</code>	Idle

The distinction between `task_abort` and `user_stop` is a deliberate accessibility decision: a voice “cancel” during a task drops back to *listening*, so the user can immediately speak the next command; only the explicit Stop gesture (or a fatal condition) returns the session to Idle, where the next interaction requires the Start button — which a blind user would have to hunt for mid-flow.

Services layer (`services/`). Speech recognition, command parsing, TTS, object detection, hand pose, spatial reasoning, and guidance phrase generation, each a stateless, individually importable module. Heavy ML imports are deferred to first use be-

hind thread-safe, double-checked lazy loaders, so importing a service — in tests, or at server start — costs nothing; the pure helpers inside them (detection parsing, landmark conversion) are directly unit-testable without any ML framework installed. The spatial-reasoning module carries the complete per-class size-prior table (Table 6.6): for each supported COCO class, the apparent size — the larger of the box’s width and height as a fraction of the frame — expected when the object sits at arm’s reach (~ 40 cm) in front of a typical phone camera.

Table 6.6: The per-class “near at arm’s reach” apparent-size priors used for distance bucketing. *near* at ≥ 0.85 of the class prior; *far* at ≤ 0.35 ; *medium* between. Unlisted classes default to 0.22 (book-sized).

Prior	Classes	Physical intuition
0.10	mouse, scissors	palm-sized
0.12	cell phone, remote	narrow handhelds
0.18	cup, bottle, clock, vase	hand-sized; bottles are tall, the $\max(w, h)$ cue covers them
0.22	book	the fallback default
0.40	keyboard, laptop, microwave, sink, toilet, chair	mid-sized objects and fixtures
0.50	oven, dining table	large appliances / furniture
0.55–0.60	TV; couch, bed, refrigerator	the largest classes — near only when they dominate the frame

Task engines. Object Allocation (which contains the reach-guidance phase) and the Navigation package, each an asynchronous per-session loop created by the FSM entry hook and cancelled by the exit hook — so a task can never outlive its state. Both entry hooks first cancel any loop already attached to the session, so even a pathological double transition cannot leak a running loop.

6.5 The Voice Pipeline

A push-to-talk release delivers one WebM/Opus blob to the audio handler, which first saves it to disk (`captured_audio/`, timestamped per session) for offline inspection, then runs:

1. **Decode.** PyAV auto-detects the container from the bytes and resamples to the 16 kHz mono float PCM Whisper expects; a resampler flush recovers the trailing samples, and a blob without an audio stream degrades to an empty transcription rather than an exception.
2. **Transcribe.** faster-whisper (`base`, `int8` quantization, beam size 5, English, no VAD — the push-to-talk gesture already trims the utterance) materializes the segments; a confidence proxy in $[0, 1]$ is derived as $e^{\overline{\log p}}$ from the average log-

probability. The transcription and confidence are pushed to the client for visual audit.

3. **Parse.** The command parser classifies the utterance as a task command, a completion (confirm / cancel), an information query (describe / repeat), or unknown.
4. **Act.** Task commands populate the task context *before* firing `command_recognized` — the FSM entry hook reads the target synchronously inside the event. Completions route by context: “got it” during a task fires `task_complete`; “cancel” during a task fires `task_abort`, dropping back to listening; “cancel” while merely listening ends the session.
5. **Confirm.** Every path ends in a spoken response — a confirmation (“Looking for your cup.”), a completion phrase naming the object, or a retry prompt — synthesized by gTTS and shipped as an MP3 binary frame.

Failures speak too: an undecodable or silent recording produces “I didn’t hear anything. Please try again,” alongside a machine-readable error — the spoken cue matters most, since a blind user has nothing to look at.

6.5.1 The Command Parser

Rather than matching whole utterances, the parser tries every (prefix, target) split of the normalized input, fuzzy-matching the prefix against the command templates and the target against the noun vocabulary with rapidfuzz’s ratio scorer, and taking the best-scoring combination over all splits; both endpoints slide, so trailing filler (“... please”) is tolerated. Normalization lowercases, strips punctuation, collapses whitespace, and applies a synonym table at word boundaries *before* matching, canonicalizing common aliases and mishearings (“fone” → cell phone, “fridge” → refrigerator, “sofa” → couch, “washroom” → bathroom) so the fuzzy thresholds do not have to be widened to absorb them. Completion and information phrases are matched with token-set similarity — so “okay, I got it, thanks” still matches “got it” — but are restricted to short utterances and high score thresholds so they can never steal a genuine find/navigate command. Table 6.7 lists the vocabulary sizes and thresholds as committed.

The thresholds are deliberately permissive at the task layer: the worst case of a false positive there is a wrong spoken confirmation the user immediately corrects, whereas a false *negative* forces a blind user to repeat themselves. Completion matching is the opposite — strict and length-limited — because a completion misfire silently ends a running task. During navigation, the same mid-task channel additionally accepts direction hints — *left*, *right*, *ahead*, *behind*, and their common variants — consumed by the multi-door dialogue of Section 5.2.9 when a scan has found more than one door.

Table 6.7: Command-parser vocabulary sizes and matching thresholds (rapidfuzz scores, 0–100).

Vocabulary	Size
Object Allocation prefixes (“find my”, “where is”, ...)	16
Navigation prefixes (“take me to”, “go to the”, ...)	10
Object nouns (COCO class names, verbatim)	21
Navigation destinations	10
Synonym / mishearing rewrites	21
Completion — confirm phrases (“got it”, “found it”, ...)	13
Completion — cancel phrases (“stop”, “never mind”, ...)	12
Info — describe phrases (“what do you see”, ...)	15
Info — repeat phrases (“say that again”, ...)	9
Threshold	Value
Prefix similarity floor	70
Noun similarity floor	60
Combined (mean) floor	65
Completion match: token-set score / max words	86 / 5
Info match: token-set score / max words	82 / 8

6.6 The Object Allocation Engine

The engine separates decision from I/O. All policy — the temporal filter, the per-class buckets, the debounce, the throttles, the scan/lost/timeout edges, and the reach state machine — lives in a `GuidanceTracker` class that is pure: it receives detections, the frame shape, an injected timestamp, the optional hand pose, and the motion state, and returns an action and a phrase (or nothing). The asynchronous shell around it only does I/O: it grabs the latest frame, runs YOLO (and, when the tracker says the target is near, MediaPipe) in worker threads so the event loop keeps serving the WebSocket, ships any pending haptic pulse *before* the phrase — a brief vibration a beat before the voice cue is a better “you’re close” signal than after — speaks the phrase, and sleeps to the 5 Hz cadence. Two actions end the loop from inside: *touch* fires `task_complete` (the auto-grasp exit), and *timeout* fires `user_stop` after the apology. Table 6.8 collects the engine’s tuning constants.

Table 6.8: Object Allocation engine constants as committed.

Constant	Value
Detection loop rate	5 Hz
YOLO confidence floor (search) / (describe query)	0.35 / 0.50
Temporal window / required hits	5 frames / 3
Spatial debounce (consecutive same-bucket frames)	2 (~ 400 ms)
Guidance re-affirm interval	6 s
Scan-prompt interval (target never seen)	8 s
Not-found timeout	60 s
Reach cue re-affirm interval	5 s
Reach grace period (hand momentarily lost)	3 s
“Almost there” zone (fingertip-to-centroid)	15% of frame

6.7 The Navigation Engine: From Prototype to Integration

The navigation subsystem has an unusual history that shaped its implementation: it was built and field-tested as a *standalone prototype* before being integrated into the backend.

6.7.1 The Standalone Prototype

The exploration pipeline was developed in a self-contained web demo: a minimal FastAPI server exposing an HTTP `/detect` endpoint, a single-page frontend that captured the rear camera and compass, POSTed one frame at a time (~ 3 per second) as a base64 data

URL, and spoke the returned guidance through the browser’s own speech synthesis. All state was module-global — one user, one journey. This shape was deliberate: it let the perception stack be iterated against real phone footage in real rooms, failure by failure, with per-candidate debug traces printed at the terminal for every keep/reject decision, without touching the main backend. The prototype survives in the repository history (branch `nav-integration`, directory `webdemo/`) as a record of that phase.

6.7.2 Integration

Once field-tested, the prototype was ported into the backend’s `services/navigation/` package. The decision logic was kept unchanged where possible, byte for byte; what changed is the plumbing:

- **Per-session state.** The module-global state dictionary became a `NavState` class, one instance per session, preserving dict-style access (`st["mode"]`) so the ported decision code reads identically to the field-tested original. Phase transitions go through `enter_*` methods — the only sanctioned way to switch phase — each of which resets exactly the fields that phase depends on (fresh approach $\Rightarrow$ fresh obstacle debounce, fresh transit latch, fresh one-shot call-out flag).
- **Frame transport.** One HTTP POST per frame became a server-driven loop over the session’s WebSocket frame stream: the engine reads the latest decoded frame each tick instead of being pushed one. One conversion happens at the boundary: the frame handler stores RGB, while the ported perception stack — and every threshold tuned against it — speaks OpenCV’s BGR, so the engine converts once per tick.
- **Speech.** Browser speech synthesis became server-side gTTS over the same binary TTS channel every other task uses; the prototype frontend’s queueing rules were re-implemented server-side as the priority/ambient policy of Section 5.2.10.
- **Model loading.** The prototype loaded all models at import. The integrated package loads lazily behind a thread-safe `ensure_loaded()`, called in a worker thread by the first navigation task in the server’s lifetime — announced to the user as “Give me a moment while I get ready” — then reused by every later session; the models are stateless and shared, the state is not. A warm-up inference on a blank frame at load time moves GPU kernel initialization off the first real frame, which would otherwise be a long silence at the start of the first journey.

6.7.3 Package Layout and the Per-Frame Pipeline

The package separates concerns strictly (Table 6.9), and `config.py` concentrates all 64 tuning constants in one file, each annotated in source with the live failure that motivated

its value — so a threshold is never a magic number, and re-tuning for a new environment touches one file.

Table 6.9: The `services/navigation/` package (2,190 lines total).

Module	Lines	Responsibility
<code>config.py</code>	~250	all 64 tuning constants, grouped by concern, values only
<code>controller.py</code>	~620	the exploration state machine and every spoken phrase (pure)
<code>perception.py</code>	~320	the door funnel, COCO stage, overlay boxes, door geometry
<code>engine.py</code>	~230	the 3 Hz async loop; motion gate; the speech policy
<code>obstacles.py</code>	~200	the walking-lane watchdog: named-object + floor layers
<code>models.py</code>	~160	thread-safe lazy loading + warm-up of the five models
<code>state.py</code>	~150	per-session <code>NavState</code> ; the <code>enter_*</code> phase resets
<code>goals.py</code>	~150	destination aliases, indicator tables, the arrival rule
<code>geometry.py</code>	~110	compass/bearing math, clustering, the $\pm 180^\circ$ seam (pure)

Each 3 Hz tick, the engine: reads the latest frame and heading off the session; converts RGB to BGR; runs the blocking perception stage in a worker thread — motion gate, COCO objects, the door funnel, door geometry, indicator-evidence accumulation, transit detection, and the obstacle watchdog — and hands the result to `controller.step()`, which returns the guidance line, its priority tier, and the arrival verdict. Priority lines are spoken immediately; ambient lines pass the repeat/gap filters first. Arrival speaks the composed summary-plus-arrival phrase and fires `task_complete`; a destination the goal model cannot resolve speaks an apology listing the supported rooms and fires `user_stop`. Cancellation — voice “stop”, the Stop button, or a disconnect — funnels through the FSM exit hook, which cancels the loop’s asyncio task. Table 6.10 collects the constants most consequential to behavior; the failure each one answers is discussed with its mechanism in Chapter 5.

6.8 Session Continuity and Haptics

Two smaller mechanisms round out the session model. **Resume.** On every connection the server sends the client its session id (`session_hello`); the client stores it in local storage. If the connection later drops with a task active, the session’s cleanup saves the task type and target into a resume pool with a five-minute TTL (pruned on access, so the pool stays bounded). On the next connect the client presents the old id; if live state exists, the

Table 6.10: Key navigation constants as committed (selection; `config.py` holds all 64).

Constant	Value
Detection loop rate	3 Hz
COCO / door confidence floors	0.45 / 0.45
Door temporal rule (window / hits)	2-of-3
Strong-door confidence (passes without corroboration)	0.60
Verifier confidence / corroboration IoU	0.30 / 0.40
Door max width/height ratio; max frame fill (scan)	1.4; 0.80
Door edge-density demotion threshold	0.030
Min door sighting units / indicator sightings per scan	3 / 4
Scan sectors; full-turn / return-to-start tolerances	$12 \times 30^\circ$; 350° / 25°
Motion-gate threshold / coach after consecutive frames	85.0 / 3
Door height / assumed HFOV / step length	2.0 m / 60° / 0.75 m
At-door: fill fraction / near distance / arming streak	0.85 / 2.0 m / 2 frames
Transit: gone cycles / min walking frames / hold cap	6 / 4 / 40
Obstacle lane (horizontal band) / min overlap	0.30–0.70 / 0.10
Obstacle debounce: blocked / clear / re-warn frames	3 / 2 / 9
Floor lane rows / band / min walkable cover	0.66–0.94 / 0.38–0.62 / 0.60
Ambient speech: min gap / identical-line repeat window	2.5 s / 8 s

server announces “Resuming search for your cup,” then re-drives the FSM through the normal start-and-recognize path so the task loop restarts identically to a fresh request. **Haptics.** The server sends symbolic pattern names that the client maps to vibration sequences (Table 6.11); iOS has no vibration API, so the client degrades silently there and voice remains the primary channel. Unknown pattern names are ignored, keeping the vocabulary extensible without client updates.

Table 6.11: The haptic pattern vocabulary.

Pattern	Vibration	Meaning
short	~40 ms tap	entered the reach “almost” zone
medium	~80 ms tap	generic acknowledgement
long	~200 ms	fingertip touched the target box
success	double tap	task completed via voice “got it”
cancel	single medium tap	task cancelled

6.9 Frontend

The client is deliberately thin: a coordinator plus six focused ES modules (Table 6.12), 1,435 lines in total including markup and styles, with no build step — the backend serves the files directly.

Table 6.12: Frontend modules.

Module	Lines	Responsibility
app.js	322	coordinator: button wiring, audio priming, PTT gestures
media.js	203	getUserMedia; 640 × 480 JPEG at 5 FPS; PTT MediaRecorder
ws_client.js	197	JSON dispatch by type; tagged binary dispatch; one auto-retry
audio_queue.js	192	the primed persistent audio element; sequential playback
motion.js	108	accelerometer window → still / moving / walking
heading.js	83	compass monitor, throttled to $\geq 2^\circ$ at ≤ 5 Hz
wakelock.js	51	screen wake lock with visibility-change reacquisition

Design decisions that matter:

- **Tap-anywhere push-to-talk.** Any pointer-down anywhere on the page (excluding the Start/Stop controls) starts recording and the matching pointer-up stops it, attached at document level so a drag off the original target still ends cleanly, with a blur handler as a safety net so a backgrounded tab never leaves the microphone hot.

The visible PTT button remains as a large tactile target and recording indicator, but a blind user never has to hunt for it.

- **iOS audio unlock.** iOS enforces audio policy at two independent layers, and TTS clips arrive well outside any user gesture. The queue therefore primes *both* inside the Start tap: it resumes a Web Audio context and plays a programmatically constructed silent WAV through one persistent `<audio>` element — which is then reused, by swapping its source, for every subsequent MP3 clip. A fresh audio element per clip is silently rejected on iPhone; the persistent primed element is what makes the voice channel work at all on iOS Safari and Chrome.
- **Sequential playback.** Clips are queued and played one at a time through that element; nothing ever overlaps on the client side.
- **Sensor permissions inside the gesture.** iOS 13+ requires motion and orientation permission requests from inside a user gesture, so both monitors start from the Start button’s click handler; either failing is non-fatal (the server falls back to plain debouncing and the compass-less scan respectively). Motion is classified from a one-second window of accelerometer deviation into still / moving / walking and sent only on state change.
- **No stale code.** The backend serves the frontend with no-cache headers, because mobile Safari’s aggressive JS caching silently reintroduced already-fixed bugs during development.
- **Local speech fallback.** Browser speech synthesis is used only where the server cannot speak — camera permission failures before the socket opens, and unexpected disconnections mid-session.

6.10 Deployment

The backend serves the frontend same-origin, so a single HTTPS endpoint exposes the whole system, and the page derives its WebSocket URL from its own origin — the same code works on localhost, behind a tunnel, or on a cloud host. For development and demos, a Cloudflare quick tunnel provides a real HTTPS URL in one command — necessary because browsers permit camera and microphone access only over HTTPS (or localhost):

```
cd backend && uvicorn main:app --port 8000
cloudflared tunnel --url http://localhost:8000
```

The phone opens the printed URL; no certificates, DNS, or router configuration are involved. Table 6.13 inventories every model the system uses, its size, its provenance, and when it loads. Public weights are fetched and cached automatically on first use; the two custom door models are committed to the repository, so a fresh clone reaches the complete working system with the two commands above and no manual asset steps.

Table 6.13: Model inventory: size, source, and load time.

Model	Size	Source	Loaded
Whisper base (CTrans-late2)	~150 MB	HuggingFace	first transcription
YOLOv8n (COCO)	~6 MB	Ultralytics CDN	first detection
MediaPipe HandLand-marker	~10 MB	Google CDN	first reach phase
YOLOv8m (COCO)	~50 MB	Ultralytics CDN	first navigation task
Door detector (YOLOv8n, 1 class)	committed	this project	first navigation task
Door verifier (YOLOv8s, 4 classes)	committed	this project	first navigation task
SegFormer-B0 (ADE20K)	~15 MB	HuggingFace	first navigation task; optional

6.11 Test Infrastructure

All decision logic is written as pure, injectable code: trackers and controllers receive detections, frame shapes, timestamps, and sensor readings as arguments and return actions and phrases — no I/O, no models, no global clock. This is what makes the automated suite possible: **281 collected test cases** across 13 modules (Table 6.14), covering FSM transitions (valid, invalid, and no-op, with subscriber semantics); command parsing across 50+ realistic transcriptions including mishearings, synonyms, filler, and the completion, cancel, and information vocabularies; spatial bucketing with per-class priors and boundary cases; guidance phrase rendering; the reach-guidance state machine including the dominant-axis rule and degenerate inputs; hand-landmark conversion; YOLO detection parsing; the Object Allocation tracker end-to-end with scripted frames and a fake clock (temporal filtering, debounce, throttles, scan/lost/timeout edges, reach entry and the once-only touch exit); the navigation goal model and arrival rules; the compass and bearing mathematics including clustering across the $\pm 180^\circ$ seam; and complete scripted navigation journeys — a full 36-heading compass scan, door selection from clustered bearings, the guided turn, the call-out and path-clear verdict, the at-door instruction, doorway transit gated on camera motion, the new room’s scan, directed indicator confirmation, and the arrival line naming the observed evidence — with the exact mode transitions and key phrases asserted at every step. Session-isolation tests verify that concurrent `NavState` instances share nothing.

The suite deliberately loads no ML model — perception results are faked at the same interfaces the engines use — and completes in under one second, so it runs on every change. The heavy perception itself (the door funnel against real pixels) is what the live trials of Chapter 7 address.

Table 6.14: Automated test suite composition (cases as collected by pytest).

Module	Cases	Subject
test_command_parser	56	task commands, mishearings, synonyms, filler
test_navigation_goals	27	aliases, indicators, the arrival rule
test_command_completion	27	confirm/cancel vocabulary and guards
test_fsm	26	every transition, rejection, and subscriber
test_spatial_reasoning	23	regions, per-class priors, boundaries
test_reach_guidance	23	touch geometry, dominant axis, degenerate input
test_command_info	22	describe/repeat vocabulary and guards
test_navigation_geometry	20	bearings, clustering, the $\pm 180^\circ$ seam
test_guidance_generator	19	every phrase template
test_object_allocation	17	the tracker end-to-end, fake clock
test_navigation_controller	8	scripted multi-room journeys
test_yolo_service	7	detection parsing (no torch)
test_hand_service	6	landmark-to-pose conversion
Total	281	

Chapter 7

Evaluation

The evaluation combines four complementary methods, chosen to match what each part of the system actually claims. Trained models are evaluated with standard detection metrics on held-out data. The voice front-end is evaluated offline against a constructed utterance corpus. Decision logic — everything the user hears, and when — is verified by automated behavioral tests that replay scripted scenarios deterministically. The full perception stack, which no offline metric can certify against real rooms, real lighting, and real clutter, was evaluated through iterative live trials on phones that directly shaped the design. We report each in turn, then trace the outcome back to the requirements of Chapter 4, and close with the system’s operational performance profile.

7.1 Door Detector Results

The single-class door detector was evaluated on its 842-image held-out validation split (Section 5.2.2). Table 7.1 reports the best checkpoint alongside the shorter training runs that preceded it, confirming the selected model.

Table 7.1: Door detector validation results (842 held-out images).

Run	mAP@50	mAP@50–95	Precision	Recall
8 epochs	0.86	—	—	—
20 epochs	0.93	—	—	—
39 epochs (selected)	0.946	0.829	0.968	0.892

Precision of 0.968 — very few false doors — is the metric that matters most for safety, and it is materially helped by the 827 hard-negative images (handles, cabinets) included in training. Recall of 0.892 means roughly one door in ten is missed in a single frame; the system’s temporal accumulation across the room scan makes the practical miss rate far lower. At the navigation loop’s 3 Hz, a slow full turn exposes the detector to on the order of a hundred frames per room, and a door needs only three sighting units across

the entire sweep — two frames if the verifier corroborates them — to become a target. Under an independence approximation, even a per-frame recall well below the validation figure leaves the probability of missing a door that stays in view for a handful of frames vanishingly small; conversely, the same accumulation is why per-frame *precision* is the number the design leans on, since false positives are the error the funnel exists to absorb. The mAP@50 of 0.946 comfortably exceeds the ≥ 0.6 target set when the milestone was planned.

An important caveat we report deliberately: the validation images come from the same public datasets as training, so these numbers measure generalization across images, not across domains. The live trials (Section 7.4) exposed domain-shift failures — most notably textured walls scoring as doors at high confidence — that the validation set cannot, which is precisely why the perception funnel of Section 5.2.3 exists: the deployed system never acts on the raw detector alone.

7.2 Offline Evaluation of the Command Parser

The command parser is the system’s single point of entry — every task begins with it — so it was evaluated offline against a constructed corpus of **533 utterances**, run through the committed parser implementation. The corpus was built systematically rather than sampled, in six blocks: the exhaustive grid of every supported noun under every Object Allocation prefix ($21 \times 16 = 336$); the exhaustive grid of every destination under every navigation prefix ($10 \times 10 = 100$); 25 noisy utterances combining transcription-style mishearings (“kichen”, “botle”, “labtop”, “refrigerator”), synonyms (“washroom”, “lounge”, “study”, “sofa”), spacing errors (“bed room”), and trailing filler; the full completion vocabulary plus filler-wrapped variants (28); the full information-query vocabulary (24); and 20 out-of-scope utterances (“what time is it”, “play some music”, “set a timer”) that the parser must classify as *unknown* rather than force into a task. An utterance counts as correct only if both the task type and the canonical target are exact. Table 7.2 reports the results.

The three failures are as informative as the successes. “Find my cup, thanks” was classified as a *completion* — the trailing “thanks” is itself a confirm phrase, and the token-set matcher scores it above the completion threshold on a five-word utterance; the guard rails (short-utterance limit, high threshold) contain the pattern but do not eliminate it when a confirm word rides on a command. A self-correction (“I need my glasses, no wait, my cup”) exceeded the tolerable edit distance for any single split and fell to *unknown* — the safe failure, since the user is re-prompted rather than sent after the wrong object. Finally, one out-of-scope utterance (“read me the news”) was force-matched to a navigation command through the permissive prefix threshold — the known cost of thresholds tuned to favor recall over precision at the task layer, where a wrong confirmation is immediately

Table 7.2: Command parser offline evaluation (533 utterances; correct = exact task type and canonical target).

Block	Correct	Accuracy
Object commands, clean (all nouns $\times$ all pre-fixes)	336/336	100.0%
Navigation commands, clean (all destinations $\times$ prefixes)	100/100	100.0%
Mishearings, synonyms, and filler	23/25	92.0%
Completion vocabulary (confirm / cancel)	28/28	100.0%
Information queries (describe / repeat)	24/24	100.0%
Out-of-scope rejection	19/20	95.0%
Overall	530/533	99.4%

audible and correctable. The exhaustive clean grids passing in full confirm the property the test suite also asserts: every supported noun and destination is reachable through every template.

7.3 Automated Behavioral Verification

The automated suite (Section 6.11) functions as a behavioral specification: each test asserts that a scripted situation produces the required action and utterance. On the submitted revision the suite passes at **281/281**, completing in **0.7 s** with no ML model loaded; the per-module composition is given in Table 6.14. Highlights relevant to the system’s claims:

- **Complete navigation journeys.** A scripted two-room journey drives the controller through a full compass scan (36 simulated headings), door selection from clustered bearings, the guided turn to face it, the one-shot door call-out followed by the path-clear verdict, the at-door instruction, doorway transit — asserted to require both door disappearance and sustained camera motion — the new room’s scan anchored to the new heading, indicator sighting, directed confirmation, and the arrival announcement naming the observed evidence (“fridge”), with the exact mode transitions and key phrases asserted at every step.
- **Safety invariants.** Tests assert that a blocking obstacle overrides door guidance with a priority warning; that an empty room produces a rescan rather than a fabricated target; that a lone sink never declares a kitchen while a refrigerator does (the primary/secondary arrival rule, exercised across all six destination types); and that arrival evidence must be localized — scan-state resets between rooms are verified so evidence can never leak across a doorway.
- **FSM discipline.** All valid transitions, all rejections (e.g., `task_complete` outside an active task), the no-op semantics of unrecognized commands, subscriber noti-

fication exactly on real transitions, and the abort path that returns to listening rather than idle — so a voice “cancel” never strands the user at a Start button they cannot see.

- **Voice robustness.** The command parser is tested against 50+ realistic transcriptions, including mishearings (“cop” → cup, “fone” → cell phone, “microvave” → microwave), synonyms (“sofa”, “fridge”, “washroom”), trailing filler, punctuation and case noise, multi-word targets (“dining table”, “living room”), and the completion, cancel, and information vocabularies used mid-task; a closing pair of tests verifies that *every* supported noun and destination is reachable through at least one template.
- **Reach guidance geometry.** The touch condition at box corners and edges, the dominant-axis rule for diagonal offsets, the almost-zone threshold, and defensive behavior on degenerate frames and off-frame fingertips.
- **Isolation.** Concurrent sessions are verified to share no navigation state, and re-entering a room-scan phase is verified to reset exactly the per-room evidence.

Because the decision layer is pure, these are not smoke tests: they replay the same code paths the live system executes, with only the perception inputs faked at the same interfaces the engines use.

7.4 Live Trials

The perception stack was evaluated the only way it can be: on phones, in real multi-room residential settings, across furniture, lighting, and clutter. These trials ran continuously throughout development, and their findings are the origin of nearly every mechanism in Chapter 5.

7.4.1 Methodology

Trials used sighted testers executing full task sessions on real phones over an HTTPS tunnel to the development server. The perception pipeline prints a per-candidate decision trace — every door candidate’s confidence, edge density, corroboration status, and keep/reject verdict, every distance estimate with its inputs, every obstacle-lane verdict — so each failure could be diagnosed from logs, addressed by a threshold change or a structural mechanism, and re-tested in the same rooms. The navigation pipeline completed full two-room journeys — scan, door approach, obstacle interruption, doorway transit, second scan, semantic arrival — end-to-end on a phone, first as the standalone prototype and again after its integration into the final backend.

7.4.2 Principal Findings and the Mechanisms They Produced

Tables 7.3 and 7.4 summarize the representative failures observed live and the mechanism each one produced, split into perception failures and guidance/speech failures. This failure-driven record is itself an evaluation result: it documents which naive designs do *not* survive contact with real rooms.

Table 7.3: Representative live *perception* failures and the mechanisms they motivated.

Observed failure	Resulting mechanism
Blank wall detected as a door at 0.84–0.9 confidence, boxed edge-to-edge	Frame-fill gate (≤ 0.80 during scan); edge-density analysis of box interiors
Plain door in dim light as smooth as a wall — edge gate rejected real doors	Edge density demotes to semantic verification instead of rejecting
Lace curtain: edge-rich, tall, door-sized — beat all geometric gates	Verifier corroboration required (door or handle seen) for weak candidates
Real door at an angle read as “refrigerator” → false kitchen evidence	Fridge arbitration layer (verifier must confirm a refrigerator door)
Verifier’s fridge veto on frames it also corroborated broke door streaks	Veto restricted to fridge evidence <i>without</i> door evidence
Whole-frame “refrigerator” on a blank wall → fake kitchen indicator	Whole-scene latch gate applied to COCO objects during scan and confirm
Scattered single-frame detections summed to false arrivals	Localized evidence: sightings must concentrate in one 90° window
Distances $\sim 33\%$ short on phones	Focal length derived from the image’s long side (portrait streaming)
Loose far-range boxes read as “near”	Distance measured on the tighter of primary and verifier boxes
Depth tripwire confused beside-path with in-path clutter	Floor segmentation replaced relative depth as the class-agnostic signal

7.4.3 Qualitative Outcomes

Beyond the individual fixes, three end-to-end behaviors were established in live runs and are worth stating as results in their own right.

Multi-room journeys complete. The full loop — guided scan, door call-out, approach, doorway transit, fresh scan in the new room, semantic arrival on the kitchen’s fridge and oven — runs end-to-end on a phone, two rooms deep, producing the session transcript shown throughout Chapter 5. The doorway transit, identified in the design phase as the riskiest step, resolves reliably once gated on camera motion: the system

Table 7.4: Representative live *guidance and speech* failures and the mechanisms they motivated.

Observed failure	Resulting mechanism
Scan ended early on compass noise; directions computed from a wrong anchor	Completion requires accumulated 350° <i>and</i> confirmed return to start
Door filed by camera heading only — announced left, was right	True bearing includes the door’s in-frame horizontal offset
Door behind the user split into “two doors behind you”	Bearing clustering merges across the $\pm 180^\circ$ seam
“At the door” fired from 2 m on a single loose box mid-sidestep	Consecutive-frame arming streak; the approach must have gotten near first
Door at arm’s length is a smooth panel — edge gate blocked the transit latch	Distance-independent “door grew to fill the frame” arming signal
“You’re through” stalled ~ 16 s by far doors of the <i>next</i> room	Far glimpses do not reset the gone-counter; a hold cap forces the inference
Detection flicker alone spoke “you’re through” while the user stood still	Movement gate: sustained camera motion is required for any transit
Announcement talked over the following instruction	Utterance merging; two-tier priority/ambient speech policy
Side furniture grazing the lane edge triggered stop warnings	Minimum lane-overlap raised; the judged lane narrowed to the walked corridor
Normal handheld turning tripped the blur gate constantly	Motion threshold retuned; coaching only after three consecutive fast frames

waits at the door while the user stands still and announces the transit shortly after they actually walk.

Obstacle interruption behaves as specified. A chair placed in the walking lane mid-approach produced the warning, the sidestep advice, the repeated prompt while still blocked, the all-clear, and re-orientation toward the door — in that order, in merged utterances, without talking over itself. The floor-segmentation layer flagged unnamed clutter (boxes, a standing fan, clothes piles — none of which COCO can name) when it lay in the walked corridor while staying silent about the same objects beside it, which is the distinction the earlier depth-based formulation failed to make.

Semantic arrival is convincing and resistant to noise. Entering a kitchen, the scan names the refrigerator and oven it confirmed and declares arrival; the localized-evidence rule visibly suppressed the false arrivals that scattered single-frame detections previously caused, and the directed double-check means a journey never ends on scan evidence the user was not shown.

The distance model deserves a specific note: after the portrait-axis correction and the tighter-box measurement, spoken step counts matched paced-out distances closely enough in trials that the one-shot calibration constant was left at its neutral value.

7.5 Evaluation Against Requirements

Table 7.5 traces each requirement from Section 4.2 to the mechanism that satisfies it and the evidence verifying it.

7.6 System Performance

The system’s responsiveness rests on a small number of measured and structural facts rather than on raw model speed.

Perception cadence and budget. Object Allocation analyses frames at 5FPS, giving each tick a 200 ms budget: one YOLOv8n inference (a ~6 MB nano model at 640 input, CPU real-time at this rate) plus, only in the reach phase, one MediaPipe hand inference measured at ~15 ms per frame during development — which is precisely why the hand model is gated on the tracker’s *near* verdict rather than run continuously. Navigation runs at 3FPS, a 333 ms budget across up to four models per frame (YOLOv8m, the two door models, and SegFormer during walking), with two structural savings: the COCO model is class-filtered to the current goal’s indicators (plus obstacle classes only while walking), and the verifier runs only when there is a door candidate or a fridge claim to check. All inference executes in worker threads, so the event loop — and with it the WebSocket, the voice pipeline, and other sessions — never blocks on a frame. Both

Table 7.5: Requirements verification. AT = automated tests; OE = offline evaluation; LT = live trials.

Req.	Satisfied by	Verified by
FR1	PTT pipeline; parser’s task/completion/info vocabularies; UI Start/Stop fallback	AT (parser, FSM); OE (99.4% on 533 utterances); LT
FR2	YOLOv8n + temporal filter + per-class spatial reasoning + guidance policy	AT (tracker end-to-end); LT
FR3	Reach phase: MediaPipe gating, fingertip geometry, auto-complete on touch	AT (reach geometry, once-only touch); LT
FR4	Exploration controller: scan, arrival rule, door funnel, transit	AT (scripted journeys); LT (two-room journeys)
FR5	Obstacle watchdog: named-object lane + floor segmentation, priority speech	AT (preemption invariant); LT (staged obstacles)
FR6	“Describe” scene readout; “repeat” replay of the last utterance	AT (info parsing, phrase rendering); OE; LT
FR7	Resume pool (5-minute TTL) + client session id + spoken resumption	LT (reconnect mid-task)
NFR1	Same-origin web frontend; tunnel HTTPS; auto-fetched weights	LT (fresh-phone deployments)
NFR2	5FPS / 3FPS loops; inference in worker threads; latest-frame reads	LT (guidance tracks scene changes)
NFR3	Two-tier speech policy; utterance merging; sequential client playback	AT (merged phrases); LT
NFR4	Temporal filters; localized evidence; the three-layer door funnel	AT (invariants); LT (Tables 7.3–7.4)
NFR5	Pure, injectable decision layer	AT: 281/281 in 0.7 s, no models loaded
NFR6	2–4 m advisory phrasing; cane-complementary guidance language	Design review; LT protocol

loops read the *latest* frame rather than a queue, so processing can never fall progressively behind the camera: under load the system skips frames, it does not lag. The motion gate adds a further saving during scans: blurred frames are skipped before any model runs, at the cost of a 64×48 grayscale difference.

Evidence timing. The scan and confirmation constants translate into concrete user-facing latencies at 3 Hz. A door needs three sighting units — two verifier-corroborated frames, or three bare strong-confidence frames — of the roughly one hundred a slow full turn provides, so a real door is virtually never missed by a completed scan, while a wall pattern must fool the funnel three separate times to become a target. An indicator needs four localized sightings to arm an arrival and three more during the directed re-check, plus a ~ 1 s settle window — so a confirmed arrival costs the user roughly two seconds of steady pointing, while a scattered artifact can never accumulate the localized evidence at all. The obstacle watchdog’s 3-blocked / 2-clear debounce means a warning follows a real blocker by ~ 1 s and an all-clear follows a sidestep by ~ 0.7 s — fast enough to precede the user’s next step at walking pace, slow enough that single-frame flicker never speaks.

Startup and first-use costs. The heavyweight navigation models load once per server lifetime, in a worker thread, announced to the user (“Give me a moment...”) rather than endured as silence, and are warmed on a blank frame so GPU initialization is not paid on the first real one. Every public model weight is fetched and cached automatically on first use (Table 6.13); the custom door weights ship in the repository, so a fresh clone reaches a working system with two commands (Section 6.10).

Speech latency. Guidance phrases are templated, so the bounded 50-entry TTS cache converts the recurring lines — confirmations, scan instructions, obstacle warnings — into cache hits after first use, removing the synthesis network round trip from the steady state. Synthesis for cache misses runs in a worker thread; detection continues underneath it. On the client, the gesture-primed persistent audio element plays clips back-to-back with no per-clip setup, and the server-side ambient throttling (no ambient line within 2.5 s of any line, no identical ambient line within 8 s) guarantees the audio queue cannot build a backlog that lags behind reality — the property that matters more to a blind user than any single clip’s latency.

Verification cost. The complete behavioral suite runs in 0.7 s with no models loaded, which in practice meant it ran on every change — the property that kept the pure decision layer correct while the perception layer was being reworked failure-by-failure.

Chapter 8

Limitations

We state the system’s limitations plainly. Several are fundamental consequences of the no-infrastructure design bet — guiding a blind user with nothing but a phone camera, a compass, and a server — rather than implementation gaps; others are scope decisions with clear extension paths (Chapter 9).

8.1 Perception and Navigation

1. **No localization.** Without SLAM, maps, or markers, the system has no notion of where it is in a building. Each room is explored on local evidence only, and the system does not remember which door it entered through or which doors it has already tried — so after a doorway transit, the entry door will be re-found by the new room’s scan, and a journey through a goal-less branch can, in principle, ping-pong between two rooms or re-enter a dead end. In practice the user’s global “stop” and their own coarse sense of the space bound this, but the system itself does not.
2. **Direction hints are shallow.** At a multi-door fork the system asks, and a spoken “left” or “right” redirects the choice — but the hint applies to that fork only, in the current room, and is forgotten the moment it is used. A hint that matches no confirmed door falls back to the autonomous pick (nearest straight ahead), and nothing is remembered across forks: the same doorway may be asked about again on a later pass, and a hint cannot yet be embedded in the initial command (“the kitchen is to the left”). Deeper hint handling and per-fork memory are staged in Chapter 9.
3. **Domain shift can defeat the door models.** The door detector’s strong validation metrics (Section 7.1) measure generalization within its public training datasets. In some real rooms, textured walls fooled all three models — the primary detector, COCO, and the verifier — simultaneously. The perception funnel contains

the damage (such candidates rarely survive the gates, and never the sighting minimums), but containment is not a cure; the durable fix is a hard-negative fine-tune on exactly such footage.

4. **Advisory guidance, not collision avoidance.** Perception runs at 3–5 FPS over a network hop; its value is advance information in the 2–4 m band — where the door is, what is in the path, which room this is. The white cane remains responsible for the immediate contact zone, and every trial kept a sighted spotter present. The obstacle watchdog is additionally armed only during the door approach — the phase in which the system has instructed the user to walk — not during scans or turns.
5. **Monocular distance is approximate.** Door distances assume a 2 m door and a nominal field of view; non-standard doors, unusual lenses, or digital zoom shift the step counts. The tighter-box measurement, the median smoothing, and the tactile endgame (“reach out with your hand”) are designed so that the final meter never depends on the estimate being exact.
6. **Compass dependence.** The 360° scan’s bearings inherit the magnetometer’s weaknesses — and magnetic interference is strongest near exactly the large appliances that define a kitchen. Compass glitches are filtered (large jumps ignored; completion requires the confirmed return to start), and a compass-less fallback exists, but the fallback loses bearing summaries and the guided turn.

8.2 System and Deployment

1. **Network and cloud dependence.** All perception and speech run server-side: a session requires connectivity for camera streaming, and speech synthesis requires reachability of the online TTS service. The phrase cache absorbs repeated lines, and the resume mechanism recovers the task after a drop, but a dead connection means a silent system — mitigated only by the client’s local speech fallback announcing the disconnection.
2. **Latency floors.** Every frame and every utterance crosses the network. The architecture hides this well in the steady state (worker threads, latest-frame reads, cached phrases), but the floor is real: time-critical reactions — a person stepping into the path at close range — are bounded by capture, transport, inference, synthesis, and playback in sequence. This reinforces, rather than merely accompanies, the cane-complementary scope.
3. **English only; curated vocabularies.** Voice interaction is English; Object Allocation covers a curated set of COCO classes; Navigation recognizes six destination room types. Rooms defined by objects outside COCO — hallways, staircases, garages — cannot be arrival targets at all, since the arrival rule needs indicator classes the detector can produce.

4. **Perception logic is validated live, not by the automated suite.** The pure decision layer is exhaustively tested, but the pixel side of the door funnel — edge densities, IoU corroboration against real frames, the segmentation lane — is validated only by the live trials. A regression there would be caught by a person, not by the test run.
5. **Demonstration-grade hardening.** Detection debug tracing is left enabled by design (it is how live failures are diagnosed) and would be disabled for production; captured push-to-talk audio is stored on the server for debugging, which is a privacy consideration a deployment would need to address; and the deployment story is a developer tunnel, not a hardened multi-tenant host.

8.3 Evaluation Scope

1. **No blind-user study.** All live trials used sighted testers. The system’s central claims about cognitive load, trust, and usability are grounded in design principles from the assistive-technology literature and in the trials’ qualitative behavior — but validating them for the target population requires a formal study with blind and low-vision participants, under an appropriate safety protocol. This is the single most important item of future work.
2. **Demonstrated scope is small-scale.** Live journeys were two rooms deep in residential settings with openable doors. Building-scale environments — long corridors, identical doors, elevators, glass — are outside the demonstrated envelope, and several (glass doors, corridors with no indicator objects) are known hard cases for the current perception vocabulary.
3. **Validation data overlaps the training distribution.** As noted in Section 7.1, the door detector’s held-out split shares its source datasets with training; cross-domain performance is characterized only by the live trials.

Chapter 9

Future Work

The limitations of Chapter 8 map directly onto a staged plan. The items below are ordered roughly by leverage: each earlier item either unblocks or de-risks the ones after it.

9.1 Deepening the Human-in-the-Loop Navigation Design

The exploration design casts the user as a *direction oracle*, and the core of that interaction is now live: at a multi-door fork the system announces the doors, asks, follows a spoken “left”/“right”/“ahead”/“behind”, and falls back to the door nearest straight ahead when the user does not know (Section 5.2.9). The next milestones deepen it:

- **Hints in the initial command.** Accept a direction embedded in the task utterance itself (“the kitchen is to the left”), so the very first scan can already be biased before any fork is reached. The parser’s direction vocabulary and the bearing mathematics are shared with the fork dialogue; what is missing is carrying the hint into the task context.
- **Richer door references.** Beyond four coarse directions: ordinal and comparative references (“the second door on the right”, “the far one”), and a spoken correction channel after guidance has started (“not that one”).
- **Come-from-door exclusion.** After a doorway transit, the entry door sits roughly 180° from the entry heading, which the compass provides nearly for free. Deprioritizing (not banning) it eliminates the two-room ping-pong that is currently the main failure mode of longer journeys — and removes the most common reason the fork question has to be asked at all.

9.2 Exploration Memory and Bounded Autonomy

Beyond hints lies coarse memory — deliberately heuristic, never a map:

- **Tried-door memory and backtracking.** Bucket each door as unvisited, entry, or exhausted; prefer unvisited, backtrack through the entry door at dead ends, and avoid exhausted branches — depth-first search over an unknown room graph, with a breadcrumb stack of entry headings enabling multi-level backtracking. The compass prerequisite is already wired.
- **Room cap with a graceful hand-off.** Bound how far exploration can wander: after a configured number of rooms, do not quit silently — let the user decide: “I’m having trouble finding the kitchen. Do you want to stop, or keep going?” The user’s “stop” is already honored globally; the cap makes the check-in proactive.

9.3 Perception Robustness

- **Hard-negative fine-tuning.** The highest-value perception fix: 30–50 photographs of the deployment environment’s walls, curtains, and wardrobes, added to the door datasets with empty labels (“nothing here”), and a short fine-tune of both door models. This teaches the models that these surfaces are *not* doors instead of fencing around the weakness with gates — and would allow some gates to be relaxed, recovering recall.
- **Distance calibration.** A single ground-truth measurement per device class locks the one-shot calibration constant; per-device camera parameters (true field of view) would remove the nominal-FOV assumption entirely.
- **Vocabulary growth.** Additional destination types (hallway, stairs, exit) require indicator classes COCO lacks; a small fine-tune for architectural classes — stairs, elevators, exit signs — would extend both navigation targets and obstacle awareness. Automated tests for the perception layer’s pure parts (edge-density and IoU logic against synthetic frames) would close the current live-only validation gap.

9.4 User Studies

The decisive evaluation is a formal study with blind and low-vision participants, run under a safety protocol (sighted spotter, cane in use, staged environments first). Beyond task-level metrics — completion rates, times, interventions — the study should target the claims the design is actually built on: that task-scoped activation lowers cognitive load relative to continuous narration, that the speech discipline earns trust, and that step-count and clock-free directional phrasing are actionable for the population. Instrument-supported measures (NASA-TLX for workload, structured interviews for trust) would

complement the task metrics. The findings should be expected to reshape the guidance phrasing at least as much as the original live trials reshaped perception.

9.5 Platform Evolution

- **Reducing network dependence.** Progressively move latency-critical pieces client-side: on-device detection for the obstacle watchdog (the one component where reaction time is safety-relevant), offline TTS, or a fully on-device mode on capable phones — keeping the server path for the heavy navigation stack.
- **Hardened deployment.** A persistent HTTPS host in place of the developer tunnel, retention policies for captured audio, debug tracing behind a runtime flag, and load validation of the per-session isolation the architecture already provides.
- **Multilingual voice.** Whisper is inherently multilingual and gTTS supports many languages; the work is in the parser’s template and synonym vocabularies — with Arabic the natural first target for the project’s context.

Chapter 10

Conclusion

This work set out to answer a specific question: can an assistive system provide reliable, context-aware guidance to blind users while minimizing cognitive load, by activating perception only for explicit tasks? Lumen answers it constructively. A blind user with nothing but an ordinary smartphone and a web address can ask for an object and be guided to it, have their hand steered onto it and released with an automatic “grasp it,” or name a destination room and be walked there door-by-door — with the system silent before the task, disciplined during it, and silent again after.

The contributions claimed in Chapter 1 hold up under the project’s own evaluation. The *task-scoped perception paradigm* is enforced structurally rather than by convention: a five-state finite state machine governs every session, every transition is an explicit event, and a task engine cannot outlive its state — cancellation, completion, disconnection, and error all funnel through the same teardown. *Goal-directed exploration* demonstrates that semantic room recognition, door detection, and a compass-anchored 360° scan can substitute for the localization a phone camera cannot provide, completing multi-room journeys with no map, no markers, and no infrastructure. The *door perception funnel* shows what safety-grade perception on commodity models requires: the custom detector reaches mAP@50 of 0.946, yet the system never acts on it raw — every spoken door survived geometric gates, semantic verification by a second model, and temporal evidence accumulation, a design forged failure-by-failure against real rooms. *Zero-install delivery* proved not merely convenient but architecturally clarifying: one WebSocket, one server process, one URL. And the *testable architecture* kept the whole decision layer honest — 281 automated tests replay complete navigation journeys, safety invariants, and fifty voice-command variants in under a second, on every change.

Beyond the artifacts, the project yields a methodological lesson worth stating plainly. In assistive perception, the adversary is not the benchmark but the world: blank walls that score as doors at 0.9 confidence, lace curtains that defeat every geometric test, real doors that read as refrigerators, compasses that lie precisely next to the appliances that define a kitchen. A system whose words a blind person will act on must therefore be

engineered so that no single model’s confidence is ever trusted alone, so that evidence must be repeated and localized before it is spoken, and so that a false positive is always treated as more expensive than a false negative — a missed door costs a rescan; an invented one points a person at a wall. Equally, the voice channel is not presentation but interface: utterances that overlap, queue up, or contradict each other are functional defects, and the speech discipline that prevents them required as much engineering as the perception it narrates.

The system’s boundaries are stated honestly in Chapter 8 — no localization or door memory yet, direction hints that reach only the current fork, an evaluation with sighted testers — and each maps onto the staged plan of Chapter 9, with deeper hint handling, exploration memory, and a formal blind-user study as the decisive next steps. What stands at the end of this project is a working demonstration that structured, task-scoped assistance — respecting the cognitive load, autonomy, and safety of blind users — can be delivered today, on hardware already in the user’s pocket, and a tested foundation on which that assistance can now grow.

References

- [1] R. R. A. Bourne, S. R. Flaxman, T. Braithwaite, *et al.*, “Magnitude, temporal trends, and projections of the global prevalence of blindness and distance and near vision impairment: A systematic review and meta-analysis,” *The Lancet Global Health*, vol. 5, no. 9, e888–e897, 2017.
- [2] H. Fernandes, P. Costa, V. Filipe, H. Paredes, and J. Barroso, “A review of assistive spatial orientation and navigation technologies for the visually impaired,” *Universal Access in the Information Society*, vol. 18, pp. 155–168, 2019.
- [3] D. Dakopoulos and N. G. Bourbakis, “Wearable obstacle avoidance electronic travel aids for blind: A survey,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 40, no. 1, pp. 25–35, 2010.
- [4] R. N. Kandalan and K. Namuduri, “Techniques for constructing indoor navigation systems for the visually impaired: A review,” *IEEE Transactions on Human-Machine Systems*, vol. 50, no. 6, pp. 492–506, 2020.
- [5] B. Li, J. P. Muñoz, X. Rong, *et al.*, “Vision-based mobile indoor assistive navigation aid for blind people,” *IEEE Transactions on Mobile Computing*, vol. 18, no. 3, pp. 702–714, 2019.
- [6] M. Martinez, A. Constantinescu, B. Schauerte, D. Koester, and R. Stiefelhagen, “Cognitive evaluation of haptic and audio feedback in short range navigation tasks,” in *Computers Helping People with Special Needs (ICCHP)*, Springer, 2014, pp. 128–135. DOI: [10.1007/978-3-319-08596-8_20](https://doi.org/10.1007/978-3-319-08596-8_20).
- [7] G. Hossain, A. S. Shaik, and M. Yeasin, “Cognitive load and usability analysis of R-MAP for people who are blind or visually impaired,” in *Proceedings of the 28th ACM International Conference on Design of Communication (SIGDOC)*, ACM, 2010, pp. 137–142. DOI: [10.1145/1878450.1878475](https://doi.org/10.1145/1878450.1878475).
- [8] S. Khan, S. Nazir, and H. U. Khan, “Analysis of navigation assistants for blind and visually impaired people: A systematic review,” in *IEEE Access*, vol. 9, 2021, pp. 26 712–26 734.
- [9] R. Girshick, “Fast r-cnn,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1440–1448.

- [10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [11] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics yolov8*, version 8.0, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [12] T.-Y. Lin, M. Maire, S. Belongie, *et al.*, “Microsoft coco: Common objects in context,” in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.
- [13] [Liu *et al.*, “[verify against february references.bib: Qualitative spatial representations for indoor assistive systems],” *[VERIFY]*, 2019.
- [14] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, “Segformer: Simple and efficient design for semantic segmentation with transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 12 077–12 090.
- [15] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, “Scene parsing through ade20k dataset,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 633–641.
- [16] L. Yang, B. Kang, Z. Huang, *et al.*, *Depth anything v2*, arXiv:2406.09414, 2024.
- [17] F. Zhang, V. Bazarevsky, A. Vakunov, *et al.*, *Mediapipe hands: On-device real-time hand tracking*, arXiv:2006.10214, 2020.
- [18] A. Graves, A.-r. Mohamed, and G. Hinton, “Speech recognition with deep recurrent neural networks,” in *2013 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2013, pp. 6645–6649.
- [19] G. Hinton, L. Deng, D. Yu, *et al.*, “Deep neural networks for acoustic modeling in speech recognition,” *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82–97, 2012.
- [20] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, *Robust speech recognition via large-scale weak supervision*, arXiv:2212.04356, 2022.
- [21] J. Li, L. Deng, Y. Gong, and R. Haeb-Umbach, “An overview of noise-robust automatic speech recognition,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 4, pp. 745–777, 2014.
- [22] T. Brown, B. Mann, N. Ryder, *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [23] G. Tur and R. De Mori, *Spoken Language Understanding: Systems for Extracting Semantic Information from Speech*. John Wiley & Sons, 2011.

- [24] J. Williams, A. Raux, D. Ramachandran, and A. Black, “The dialog state tracking challenge,” in *Proceedings of the SIGDIAL 2013 Conference*, 2013, pp. 404–413.
- [25] J. Preece, Y. Rogers, and H. Sharp, *Interaction Design: Beyond Human-Computer Interaction*, 4th ed. John Wiley & Sons, 2015.
- [26] Y. Wang, R. J. Skerry-Ryan, D. Stanton, *et al.*, “Tacotron: Towards end-to-end speech synthesis,” in *Proceedings of Interspeech 2017*, 2017, pp. 4006–4010.
- [27] H. Zen, K. Tokuda, and A. W. Black, “Statistical parametric speech synthesis,” *Speech Communication*, vol. 51, no. 11, pp. 1039–1064, 2009.
- [28] Google, *Google text-to-speech*, Accessed 2026, 2023. [Online]. Available: <https://cloud.google.com/text-to-speech>.
- [29] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [30] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018.
- [31] L. A. Guerrero, F. Vasquez, and S. F. Ochoa, “An indoor navigation system for the visually impaired,” in *Sensors*, vol. 12, 2012, pp. 8236–8258.
- [32] A. Rodríguez, J. J. Yebes, P. F. Alcantarilla, L. M. Bergasa, J. Almazán, and A. Cella, “Assisting the visually impaired: Obstacle detection and warning system by acoustic feedback,” in *Sensors*, vol. 12, 2012, pp. 17 476–17 496.
- [33] D. Ahmetovic, C. Gleason, C. Ruan, K. Kitani, H. Takagi, and C. Asakawa, “Navcog: A navigational cognitive assistant for the blind,” in *Proceedings of the 18th International Conference on Human-Computer Interaction with Mobile Devices and Services (MobileHCI)*, 2016, pp. 90–99.
- [34] A. Brilhault, S. Kammoun, O. Gutierrez, P. Truillet, and C. Jouffrais, “Fusion of artificial vision and gps to improve blind pedestrian positioning,” in *2011 4th IFIP International Conference on New Technologies, Mobility and Security*, 2011, pp. 1–5.
- [35] R. Tapu, B. Mocanu, and T. Zaharia, “Wearable assistive devices for visually impaired: A state of the art survey,” *Pattern Recognition Letters*, vol. 137, pp. 37–52, 2020.
- [36] C. D. Wickens, “Multiple resources and mental workload,” *Human Factors*, vol. 50, no. 3, pp. 449–455, 2008.
- [37] X. Chen, J. Yang, and Z. Zhang, “Wearable sensor-based assistive system for visually impaired people,” *Sensors*, vol. 15, no. 8, pp. 17 547–17 567, 2015. DOI: [10.3390/s150817547](https://doi.org/10.3390/s150817547).

- [38] L. Porzi, S. Messelodi, C. M. Modena, and E. Ricci, “A smart watch-based gesture recognition system for assisting people with visual impairments,” in *Proceedings of the 3rd ACM International Workshop on Interactive Multimedia on Mobile & Portable Devices*, 2013, pp. 19–24.
- [39] M. Arduengo, C. Torras, and L. Sentis, “Robust and adaptive door operation with a mobile robot,” *Intelligent Service Robotics*, vol. 14, pp. 409–425, 2021, Source of the DoorDetect dataset.
- [40] J. G. Ramôa, V. Lopes, L. A. Alexandre, and S. Mogo, “Real-time 2d–3d door detection and state classification on a low-power device,” *SN Applied Sciences*, vol. 3, no. 5, 2021, Source of the DeepDoors2 dataset.